

PyTorch 面试速通

最快上手 · 高频面试题 · 冲刺速成

不啃大部头，只学面试真考的

张量 · 训练 · 网络 · 分布式 · 面试八股

SZY

2026 年 6 月 23 日

前言

本书面向有 **Python** 基础、需要在短时间内冲刺面试的读者。它不是 PyTorch 大部头教程，而是一份尽量简单、精炼、高频的速通指南：只讲面试真考、实战够用的内容，帮你快速入门到能应付面试。

怎么用这本书：

- 时间紧：直接看每章末的「面试速记卡」，再扫一遍第 4、5、7 章的高频题。
- 有几天：从第 1 章顺序读到第 8 章，跟着敲一遍最小可跑代码。
- 面试前一晚：只翻第 7 章（速查表 + 坑）和各章速记卡。

深度取舍：砍掉了论文复现、手写 autograd 引擎等深水区；但保留了 LLM 时代**面试高频的分布式训练**（DDP / FSDP / 并行策略，见第 6 章）。想深入 AI Agent 应用层，可参阅同仓库的《AI Agent 工程师转行与面试完全指南》。

代码基于 *PyTorch 2.x* 现代 *API*，力求可运行、可复现。

目录

前言	2
1 快速上手：环境、张量与 autograd	4
1.1 本章定位	4
1.2 环境与 GPU 安装	4
1.2.1 pip 与 conda 安装	4
1.2.2 查版本、查 GPU、选 device	4
1.3 张量基础	5
1.3.1 创建张量	5
1.3.2 dtype 与转换	5
1.3.3 shape、索引与切片	5
1.3.4 广播规则	6
1.3.5 reshape、view、permute 与 transpose	6
1.3.6 CPU 到 GPU 迁移	7
1.3.7 与 NumPy 互转	7
1.3.8 常用算子	8
1.4 autograd 自动求导	8
1.4.1 requires_grad、计算图与叶子张量	8
1.4.2 backward、grad 与梯度累加	9
1.4.3 no_grad、detach 与 in-place 风险	9
1.5 高频面试题	10
2 一章学会建模与训练	12
2.1 nn.Module：自定义模型	12
2.1.1 parameters、named_parameters 与 device	12
2.1.2 train 与 eval	13
2.2 常用层速览	13
2.2.1 Linear、Conv2d 与激活函数	13
2.2.2 nn.Sequential	13
2.3 损失函数	14
2.3.1 MSELoss	14

2.3.2	CrossEntropyLoss	14
2.3.3	BCEWithLogitsLoss	14
2.4	优化器与学习率调度	15
2.5	Dataset 与 DataLoader	15
2.6	完整训练循环模板	16
2.7	面试高频问答	17
2.8	章末速记	19
3	必会网络速查: MLP / CNN / RNN / Transformer	20
3.1	本章定位	20
3.2	MLP: 全连接网络	20
3.2.1	结构与直觉	20
3.2.2	最小 MLP 代码	20
3.3	CNN: 局部特征提取器	21
3.3.1	Conv2d 参数与输出尺寸	21
3.3.2	池化、感受野与参数量	21
3.3.3	1×1 卷积	21
3.3.4	最小 CNN 代码	22
3.3.5	LeNet 思路	22
3.3.6	ResNet 残差块	22
3.4	RNN / LSTM / GRU: 序列建模	23
3.4.1	RNN 与梯度消失	23
3.4.2	LSTM 的三个门	23
3.4.3	GRU 简化	23
3.4.4	<code>nn.LSTM</code> 最小用法	23
3.5	Transformer: 注意力与并行序列建模	24
3.5.1	Self-Attention 公式	24
3.5.2	为何除以 $\sqrt{d_k}$	24
3.5.3	Multi-Head、位置编码与结构	24
3.5.4	相比 RNN 的优势	24
3.5.5	<code>nn.MultiheadAttention</code> 最小示例	25
3.6	面试题速答	25
3.7	章末速记卡	27
4	高频理论面试题速记	28
4.1	反向传播与梯度直觉	28
4.2	梯度消失与梯度爆炸	29
4.3	激活函数对比	30
4.4	BatchNorm 与 LayerNorm	30
4.5	Dropout、过拟合与欠拟合	32

4.6	权重初始化	33
4.7	优化器	33
4.8	学习率	34
4.9	损失函数	35
4.10	综合追问	36
5	高频编码面试题	38
5.1	手写 Scaled Dot-Product Attention	38
5.2	手写 Multi-Head Attention	39
5.3	用 <code>nn.Parameter</code> 手写 Linear 层	40
5.4	手写 BatchNorm1d	41
5.5	手写 Dropout	42
5.6	手写交叉熵损失	43
5.7	手写完整训练循环	44
5.8	张量操作题：不用 Python for 循环	45
5.9	可选：Label Smoothing 交叉熵	46
6	分布式训练面试速通	48
6.1	为什么需要分布式训练	48
6.1.1	三个核心动机	48
6.2	DP vs DDP	48
6.2.1	DataParallel 的问题	48
6.2.2	DDP 的优势	49
6.3	DDP 原理与最小代码骨架	49
6.3.1	基本 workflow	49
6.3.2	all-reduce 与 ring all-reduce 直觉	50
6.3.3	DistributedSampler 与 SyncBatchNorm	50
6.3.4	最小 DDP 代码骨架	50
6.4	FSDP: Fully Sharded Data Parallel	51
6.4.1	FSDP 分片了什么	51
6.4.2	FSDP 与 ZeRO	51
6.4.3	auto wrap 与混合精度	52
6.5	三种并行方式	52
6.5.1	数据并行	52
6.5.2	张量并行	52
6.5.3	流水线并行	52
6.5.4	如何选择	53
6.6	混合精度 AMP	53
6.6.1	为什么能省显存和提速	53
6.6.2	最小 AMP 代码	53

6.7	显存优化常见手段	54
6.7.1	梯度累积	54
6.7.2	梯度检查点	54
6.7.3	参数 offload	54
6.8	高频面试题	55
6.9	章末速记卡	56
7	速查表与常见坑	57
7.1	常用张量操作速查	57
7.1.1	创建、形状、索引、拼接与规约	57
7.1.2	形状操作对照	58
7.2	模型层、损失、优化器与调度器	58
7.2.1	常用层速查	58
7.2.2	损失函数对照	58
7.2.3	优化器与调度器对照	58
7.3	设备迁移、保存加载与 API 对照	60
7.3.1	设备迁移与保存加载	60
7.3.2	NumPy 与 Torch API 对照	60
7.4	常见坑与排查方式	60
7.5	临场速记：面试前五分钟	62
7.6	章末 quickcard	63
8	面试打法与冲刺路线	64
8.1	本章定位：把知识变成可录用的表达	64
8.2	项目怎么讲：从流水账到结构化叙事	64
8.2.1	STAR 框架	64
8.2.2	问题-数据-模型-指标-优化-上线	65
8.2.3	把一个深度学习项目讲清楚的模板	65
8.2.4	示例话术：图像缺陷分类项目	65
8.2.5	常见追问与回答要点	66
8.3	简版机器学习系统设计答法	67
8.3.1	面试中的七个模块	67
8.3.2	例子一：设计一个图像分类服务	67
8.3.3	例子二：设计一个推荐打分服务	68
8.4	行为面试：用 STAR 回答软技能问题	68
8.4.1	常见问题与答法要点	68
8.4.2	行为题示例话术	68
8.5	冲刺学习路线	69
8.5.1	3 天速成计划	69
8.5.2	7 天速成计划	70

目录	7
8.6 资源清单：少而精	70
8.6.1 官方文档与关键 API	70
8.6.2 刷题方向	71
8.7 与《AI Agent 工程师转行与面试完全指南》的衔接	71
8.8 章末 quickcard：面试当天行动清单	71

Chapter 1

快速上手：环境、张量与 autograd

1.1 本章定位

PyTorch 入门面试题高频考三类能力：环境能否跑通，张量 API 是否熟练，autograd 是否理解。本章用精炼示例覆盖安装、GPU、张量、NumPy 互转和自动求导。

1.2 环境与 GPU 安装

1.2.1 pip 与 conda 安装

PyTorch 2.x 的安装命令应以官网为准，因为系统、Python 版本、CUDA 版本都会影响安装包。常见 pip 写法如下：

```
1 python -m pip install torch torchvision torchaudio
```

常见 conda 写法如下：

```
1 conda install pytorch torchvision torchaudio -c pytorch
```

如果要使用 NVIDIA GPU，需要显卡驱动、PyTorch CUDA 构建和运行时 GPU 可见性都正确。很多 PyTorch wheel 已自带 CUDA 运行时库，本机不一定要额外安装完整 CUDA Toolkit。

1.2.2 查版本、查 GPU、选 device

安装后先运行：

```
1 import torch
2 print(torch.__version__)
3 print(torch.version.cuda)
4 print(torch.cuda.is_available())
5 print(torch.cuda.device_count())
```

标准 device 写法如下：

```
1 import torch
2 device = "cuda" if torch.cuda.is_available() else "cpu"
```



```
3 x = torch.randn(2, 3, device=device)
4 print(x.device)
```

常见坑

`torch.version.cuda` 表示 PyTorch 构建对应的 CUDA 版本, 不代表机器一定有可用 GPU。运行时判断 GPU, 应使用 `torch.cuda.is_available()`。

1.3 张量基础

1.3.1 创建张量

常见创建函数包括 `torch.tensor`、`zeros`、`ones`、`randn` 和 `arange`:

```
1 import torch
2 a = torch.tensor([[1, 2], [3, 4]])
3 z = torch.zeros(2, 3)
4 o = torch.ones(2, 3)
5 r = torch.randn(2, 3)
6 t = torch.arange(0, 6)
7 print(a)
8 print(z.shape, o.shape, r.dtype, t)
```

`torch.tensor` 会根据输入推断 `dtype`。训练中常用 `torch.float32` 表示特征和权重, 分类标签常用 `torch.long`。

1.3.2 dtype 与转换

`dtype` 影响数值范围、显存占用和算子行为:

```
1 import torch
2 x = torch.tensor([1, 2, 3])
3 y = x.float()
4 label = torch.tensor([0, 1, 2], dtype=torch.long)
5 half_y = y.to(torch.float16)
6 print(x.dtype, y.dtype, label.dtype, half_y.dtype)
```

常用转换: `x.float()`、`x.long()`、`x.to(dtype=torch.float16)`、`x.to(device)`。

1.3.3 shape、索引与切片

形状可用 `shape` 或 `size()` 查看, 索引和切片接近 NumPy:

```
1 import torch
2 x = torch.arange(12).reshape(3, 4)
3 print(x.shape)
4 print(x.size())
5 print(x[0])
```

```
6 print(x[:, 1])
7 print(x[1:3, 2:])
```

切片通常返回 view，可能与原张量共享底层存储。

1.3.4 广播规则

广播从最后一个维度向前比较：

1. 维度相等，可以广播。
2. 其中一个维度为 1，可以扩展。
3. 维度更少的一方，在前面补 1 再比较。
4. 既不相等也没有 1，则不能广播。

例子：

```
1 import torch
2 x = torch.ones(2, 3)
3 b = torch.tensor([10.0, 20.0, 30.0])
4 y = x + b
5 print(y)
6 print(y.shape)
```

这里 x 是 (2,3)， b 是 (3)，可看成 (1,3)，再扩展到 (2,3)。`keepdim=True` 常用于保留规约维度，方便后续广播：

```
1 import torch
2 x = torch.randn(4, 3)
3 mean = x.mean(dim=0, keepdim=True)
4 std = x.std(dim=0, keepdim=True)
5 y = (x - mean) / (std + 1e-6)
6 print(mean.shape, y.shape)
```

1.3.5 reshape、view、permute 与 transpose

常见形状操作如下：

```
1 import torch
2 x = torch.arange(12)
3 a = x.reshape(3, 4)
4 b = a.view(2, 6)
5 c = a.transpose(0, 1)
6 d = a.reshape(1, 3, 4).permute(0, 2, 1)
7 print(a.shape, b.shape, c.shape, d.shape)
```

`view` 要求张量 contiguous; `reshape` 会尽量返回 `view`, 必要时复制。`transpose` 交换两个维度, `permute` 可重排多个维度。

```
1 import torch
2 x = torch.arange(12).reshape(3, 4)
3 y = x.t()
4 print(y.is_contiguous())
5 z = y.contiguous().view(12)
6 print(z)
```

常见坑

`transpose` 和 `permute` 常返回非 contiguous 张量, 之后直接 `view` 可能报错。稳妥写法是先 `contiguous()` 再 `view`, 或直接用 `reshape`。

1.3.6 CPU 到 GPU 迁移

张量和模型必须在同一个 device 上计算:

```
1 import torch
2 device = "cuda" if torch.cuda.is_available() else "cpu"
3 x = torch.randn(2, 3)
4 x = x.to(device)
5 y = torch.ones(2, 3, device=device)
6 z = x + y
7 print(z.cpu().device)
```

`x.to(device)` 通常返回新张量; 不接返回值, 原 `x` 可能仍在原 device 上。

1.3.7 与 NumPy 互转

CPU 张量和 NumPy 数组可以互转, 且通常共享内存:

```
1 import torch
2 x = torch.tensor([1.0, 2.0, 3.0])
3 a = x.numpy()
4 a[0] = 100.0
5 print(x)
6 y = torch.from_numpy(a)
7 y[1] = 200.0
8 print(a)
```

GPU 张量转 NumPy 前要先到 CPU, 训练结果通常还要先 `detach()`:

```
1 import torch
2 device = "cuda" if torch.cuda.is_available() else "cpu"
3 x = torch.randn(3, device=device)
4 a = x.detach().cpu().numpy()
5 print(a)
```

常见坑

NumPy 互转共享内存很高效，但修改一方会影响另一方。如果需要独立副本，可用 `x.detach().cpu().clone().numpy()` 或 NumPy 的 `copy()`。

1.3.8 常用算子

矩阵乘法、规约、拼接和维度增删是高频操作：

```
1 import torch
2 a = torch.randn(2, 3)
3 b = torch.randn(3, 4)
4 c = a @ b
5 d = torch.matmul(a, b)
6 x = torch.randn(2, 3, 4)
7 s = x.sum()
8 m = x.mean(dim=1)
9 v, idx = x.max(dim=2)
10 p = torch.randn(2, 3)
11 q = torch.randn(2, 3)
12 cat0 = torch.cat([p, q], dim=0)
13 stack0 = torch.stack([p, q], dim=0)
14 u = p.unsqueeze(0)
15 sq = u.squeeze(0)
16 print(c.shape, d.shape)
17 print(s.shape, m.shape, v.shape, idx.shape)
18 print(cat0.shape, stack0.shape, u.shape, sq.shape)
```

`cat` 沿已有维度拼接；`stack` 新增一个维度后堆叠。

1.4 autograd 自动求导

1.4.1 requires_grad、计算图与叶子张量

设置 `requires_grad=True` 后，PyTorch 会记录可微运算并构建动态计算图：

```
1 import torch
2 x = torch.tensor([1.0, 2.0, 3.0], requires_grad=True)
3 y = x * x + 2.0
4 loss = y.mean()
5 print(loss)
6 print(loss.grad_fn)
```

`x` 是叶子张量，`y` 和 `loss` 是运算产生的非叶子张量。反向传播后，梯度通常保存在叶子张量的 `grad` 字段中。

1.4.2 backward、grad 与梯度累加

标量 loss 可直接调用 backward():

```
1 import torch
2 x = torch.tensor([1.0, 2.0, 3.0], requires_grad=True)
3 loss = (x * x).sum()
4 loss.backward()
5 print(x.grad)
```

这里 $loss = x_1^2 + x_2^2 + x_3^2$, 所以 $\partial loss / \partial x_i = 2x_i$ 。如果输出不是标量, 需要传入同形状上游梯度。PyTorch 梯度默认累加, 因此训练循环要清零:

```
1 import torch
2 model = torch.nn.Linear(3, 1)
3 optimizer = torch.optim.SGD(model.parameters(), lr=0.1)
4 x = torch.randn(4, 3)
5 y = torch.randn(4, 1)
6 pred = model(x)
7 loss = torch.nn.functional.mse_loss(pred, y)
8 optimizer.zero_grad()
9 loss.backward()
10 optimizer.step()
```

1.4.3 no_grad、detach 与 in-place 风险

with torch.no_grad() 临时关闭梯度记录, 常用于验证和推理:

```
1 import torch
2 x = torch.tensor([1.0, 2.0, 3.0], requires_grad=True)
3 with torch.no_grad():
4     y = x * 2.0
5 print(y.requires_grad)
```

detach() 返回从当前计算图分离的新张量:

```
1 import torch
2 x = torch.tensor([1.0, 2.0, 3.0], requires_grad=True)
3 y = x * 2.0
4 z = y.detach()
5 print(y.requires_grad)
6 print(z.requires_grad)
```

in-place 操作会直接修改原张量, 常以末尾下划线表示, 例如 zero_()。它可能破坏 autograd 保存的中间值, 因此对需要梯度的张量要谨慎。

1.5 高频面试题

面试题

`view` 和 `reshape` 有什么区别？什么时候 `view` 会失败？

参考答案

`view` 只改变形状视图，要求底层内存 contiguous；非 contiguous 张量直接 `view` 可能失败。`reshape` 会尽量返回 `view`，做不到时会复制数据。一句话：`view` 更严格，`reshape` 更省心但可能有拷贝。

面试题

PyTorch 的广播规则是什么？

参考答案

从最后一维向前比较。维度相等，或其中一个为 1，就能广播；缺失的前置维度按 1 处理。例如 (2,3) 与 (3) 可看成 (2,3) 与 (1,3)，结果为 (2,3)。

面试题

`requires_grad`、`detach` 和 `no_grad` 有什么区别？

参考答案

`requires_grad=True` 表示需要追踪梯度；`detach()` 把已有结果从当前计算图分离；`torch.no_grad()` 在代码块内暂停记录新运算，常用于推理。

面试题

为什么 `numpy()` 与 `tensor` 可能共享内存？

参考答案

CPU `tensor` 调用 `numpy()` 时通常不复制数据，而是与 NumPy 数组共享底层内存以提升效率。风险是修改一方会影响另一方。需要副本时使用 `clone()` 或 NumPy 的 `copy()`。

面试题

标量 `.item()` 有什么作用？

参考答案

`.item()` 把单元素 `tensor` 转成 Python 数值，常用于日志和打印。它只能用于单元素张量；若张量在 GPU 上，调用 `.item()` 会触发 CPU 与 GPU 同步，频繁调用会拖慢训练。

面试速记卡

- 查版本: `torch.__version__`; 查 CUDA 构建: `torch.version.cuda`; 查 GPU: `torch.cuda.is_available()`; 标准 device: `device = "cuda" if torch.cuda.is_available() else "cpu"`。
- 广播从后往前比; 相等或有一个为 1 才兼容; `view` 要求 contiguous, `reshape` 更通用但可能复制。
- `cat` 沿已有维度拼接; `stack` 新增维度堆叠; CPU tensor 与 NumPy 通常共享内存。
- `requires_grad=True` 追踪梯度; 梯度默认累加, 训练循环中要 `optimizer.zero_grad()`。
- `torch.no_grad()` 关闭记录; `detach()` 分离已有计算图; `.item()` 会取 Python 标量并可能触发 GPU 同步。

Chapter 2

一章学会建模与训练

本章把 PyTorch 建模与训练的高频知识压成一条主线：写模型、选层、选损失、接优化器、喂数据，最后套标准训练循环。面试时只要抓住这条链路，多数问题都能回答得清楚：数据进入模型，模型输出 logits，损失计算标量，反向传播产生梯度，优化器根据梯度更新参数。

2.1 nn.Module：自定义模型

自定义模型通常继承 `nn.Module`。在 `__init__` 中声明层，在 `forward` 中描述张量流动。调用 `model(x)` 时会自动执行 `forward`，不要手动写 `model.forward(x)`。

```
1 import torch
2 from torch import nn
3 class MLP(nn.Module):
4     def __init__(self, in_dim, hidden_dim, num_classes):
5         super().__init__()
6         self.fc1 = nn.Linear(in_dim, hidden_dim)
7         self.act = nn.ReLU()
8         self.drop = nn.Dropout(p=0.2)
9         self.fc2 = nn.Linear(hidden_dim, num_classes)
10    def forward(self, x):
11        x = self.fc1(x)
12        x = self.act(x)
13        x = self.drop(x)
14        return self.fc2(x)
15 model = MLP(20, 64, 3)
16 x = torch.randn(8, 20)
17 logits = model(x)
18 print(logits.shape)
```

2.1.1 parameters、named_parameters 与 device

`model.parameters()` 返回模型参数迭代器，常直接传给优化器。`model.named_parameters()` 会同时返回参数名，适合调试、冻结层或给不同层设置不同学习率。


```

1 for name, param in model.named_parameters():
2     print(name, tuple(param.shape), param.requires_grad)
3 optimizer = torch.optim.AdamW(model.parameters(), lr=1e-3)

```

模型和数据必须在同一设备。标准写法是先确定 device，再把模型、输入和 target 都移动过去。

```

1 device = torch.device("cuda" if torch.cuda.is_available() else "cpu")
2 model = MLP(20, 64, 3).to(device)
3 x = torch.randn(8, 20).to(device)
4 y = torch.randint(0, 3, (8,), device=device)
5 logits = model(x)

```

2.1.2 train 与 eval

`model.train()` 与 `model.eval()` 切换模块行为，不负责开关梯度。它们主要影响 Dropout 和 BatchNorm；关闭梯度应使用 `torch.no_grad()`。

模式	Dropout	BatchNorm
<code>train()</code>	随机丢弃神经元	使用当前 batch 统计量并更新 running 统计量
<code>eval()</code>	关闭随机丢弃	使用训练期累计的 running 统计量

因此训练循环中先写 `model.train()`；验证和测试时写 `model.eval()`，并把推理代码包在 `with torch.no_grad():` 中。

2.2 常用层速览

2.2.1 Linear、Conv2d 与激活函数

`nn.Linear(in_features, out_features)` 是全连接层，输入最后一维必须等于 `in_features`。`nn.Conv2d` 常用于图像，输入形状是 `[N, C, H, W]`。常见激活有 `nn.ReLU`、`nn.GELU`、`nn.Sigmoid` 和 `nn.Tanh`。

```

1 linear = nn.Linear(128, 10)
2 x = torch.randn(32, 128)
3 print(linear(x).shape)
4 conv = nn.Conv2d(3, 16, kernel_size=3, padding=1)
5 image = torch.randn(4, 3, 32, 32)
6 print(conv(image).shape)

```

2.2.2 nn.Sequential

`nn.Sequential` 适合顺序堆叠网络层。若模型有多输入、多分支、条件逻辑或跳连，应直接在 `forward` 中写清楚。

```
1 cnn = nn.Sequential(  
2     nn.Conv2d(3, 16, kernel_size=3, padding=1),  
3     nn.ReLU(),  
4     nn.MaxPool2d(kernel_size=2),  
5     nn.Conv2d(16, 32, kernel_size=3, padding=1),  
6     nn.ReLU(),  
7     nn.AdaptiveAvgPool2d((1, 1)),  
8     nn.Flatten(),  
9     nn.Linear(32, 10),  
10 )
```

2.3 损失函数

2.3.1 MSELoss

`nn.MSELoss` 常用于回归，模型输出和 `target` 通常形状一致。

```
1 criterion = nn.MSELoss()  
2 pred = torch.randn(16, 1)  
3 target = torch.randn(16, 1)  
4 loss = criterion(pred, target)
```

2.3.2 CrossEntropyLoss

`nn.CrossEntropyLoss` 用于单标签多分类。它内部已经包含 `LogSoftmax` 和负对数似然损失，所以输入必须是 logits，不要手动 softmax。`target` 是类别索引，形状通常为 `[N]`，类型为 `torch.long`，不是 one-hot。

```
1 criterion = nn.CrossEntropyLoss()  
2 logits = torch.randn(8, 5)           # raw scores, not probabilities  
3 target = torch.tensor([0, 1, 4, 3, 2, 1, 0, 4])  
4 loss = criterion(logits, target)
```

2.3.3 BCEWithLogitsLoss

`nn.BCEWithLogitsLoss` 用于二分类或多标签分类，内部包含 `Sigmoid` 和二元交叉熵，比先 `sigmoid` 再 `BCELoss` 更稳定。`target` 通常是与 logits 同形状的浮点张量。

```
1 criterion = nn.BCEWithLogitsLoss()  
2 logits = torch.randn(8, 4)  
3 target = torch.randint(0, 2, (8, 4)).float()  
4 loss = criterion(logits, target)
```

2.4 优化器与学习率调度

`optim.SGD` 简单稳定，配合 `momentum` 常用于视觉任务；`Adam` 自适应调整每个参数的步长，收敛初期通常更快；`AdamW` 将 `weight decay` 与 `Adam` 更新解耦，是 Transformer 等模型的常用选择。`weight_decay` 用来抑制参数过大，降低过拟合。

```
1 optimizer = torch.optim.SGD(model.parameters(), lr=0.1, momentum=0.9)
2 optimizer = torch.optim.Adam(model.parameters(), lr=1e-3)
3 optimizer = torch.optim.AdamW(model.parameters(), lr=1e-3, weight_decay=0.01)
```

`StepLR` 按固定间隔衰减学习率；`CosineAnnealingLR` 按余弦曲线平滑下降。`warmup` 指训练初期从小学习率逐步升到目标学习率，常用于大模型或大 batch 训练。

```
1 optimizer = torch.optim.AdamW(model.parameters(), lr=1e-3)
2 scheduler = torch.optim.lr_scheduler.StepLR(optimizer, step_size=10, gamma=0.1)
3 for epoch in range(30):
4     train_one_epoch()
5     scheduler.step()
6 cosine = torch.optim.lr_scheduler.CosineAnnealingLR(optimizer, T_max=30)
```

2.5 Dataset 与 DataLoader

自定义 `Dataset` 至少实现 `__len__` 和 `__getitem__`。`DataLoader` 负责 batch 化、打乱、并行加载和拼接样本。

```
1 from torch.utils.data import Dataset, DataLoader
2 class TensorDataset2D(Dataset):
3     def __init__(self, x, y):
4         self.x = x
5         self.y = y
6     def __len__(self):
7         return len(self.x)
8     def __getitem__(self, index):
9         return self.x[index], self.y[index]
10 dataset = TensorDataset2D(torch.randn(100, 20), torch.randint(0, 3, (100,)))
11 loader = DataLoader(dataset, batch_size=32, shuffle=True, num_workers=0)
```

参数	作用
<code>batch_size</code>	每个 batch 的样本数
<code>shuffle</code>	训练集通常为 <code>True</code> ，验证集通常为 <code>False</code>
<code>num_workers</code>	子进程加载数据的数量
<code>drop_last</code>	是否丢弃最后一个不足 batch 的小批次
<code>collate_fn</code>	自定义样本如何拼成 batch，常用于变长序列

2.6 完整训练循环模板

标准五步是 `optimizer.zero_grad()` $\rightarrow$ `forward` $\rightarrow$ `loss` $\rightarrow$ `loss.backward()` $\rightarrow$ `optimizer.step()`。下面代码可直接运行，模拟一个三分类任务。

```
1 import torch
2 from torch import nn
3 from torch.utils.data import DataLoader, Dataset
4 class ToyDataset(Dataset):
5     def __init__(self, n=512, in_dim=20, num_classes=3):
6         self.x = torch.randn(n, in_dim)
7         w = torch.randn(in_dim, num_classes)
8         self.y = (self.x @ w).argmax(dim=1)
9     def __len__(self):
10         return len(self.x)
11     def __getitem__(self, index):
12         return self.x[index], self.y[index]
13 class Classifier(nn.Module):
14     def __init__(self):
15         super().__init__()
16         self.net = nn.Sequential(
17             nn.Linear(20, 64),
18             nn.ReLU(),
19             nn.Dropout(p=0.1),
20             nn.Linear(64, 3),
21         )
22     def forward(self, x):
23         return self.net(x)
24 def train_one_epoch(model, loader, criterion, optimizer, device):
25     model.train()
26     total_loss, total_correct, total_count = 0.0, 0, 0
27     for x, y in loader:
28         x, y = x.to(device), y.to(device)
29         optimizer.zero_grad()
30         logits = model(x)
31         loss = criterion(logits, y)
32         loss.backward()
33         optimizer.step()
34         batch_size = x.size(0)
35         total_loss += loss.item() * batch_size
36         total_correct += (logits.argmax(dim=1) == y).sum().item()
37         total_count += batch_size
38     return total_loss / total_count, total_correct / total_count
39 def evaluate(model, loader, criterion, device):
40     model.eval()
41     total_loss, total_correct, total_count = 0.0, 0, 0
```

```
42     with torch.no_grad():
43         for x, y in loader:
44             x, y = x.to(device), y.to(device)
45             logits = model(x)
46             loss = criterion(logits, y)
47             batch_size = x.size(0)
48             total_loss += loss.item() * batch_size
49             total_correct += (logits.argmax(dim=1) == y).sum().item()
50             total_count += batch_size
51     return total_loss / total_count, total_correct / total_count
52 def main():
53     device = torch.device("cuda" if torch.cuda.is_available() else "cpu")
54     train_loader = DataLoader(ToyDataset(512), batch_size=32, shuffle=True)
55     val_loader = DataLoader(ToyDataset(128), batch_size=64, shuffle=False)
56     model = Classifier().to(device)
57     criterion = nn.CrossEntropyLoss()
58     optimizer = torch.optim.AdamW(model.parameters(), lr=1e-3, weight_decay=0.01)
59     scheduler = torch.optim.lr_scheduler.StepLR(optimizer, step_size=5, gamma=0.5)
60     for epoch in range(10):
61         train_loss, train_acc = train_one_epoch(
62             model, train_loader, criterion, optimizer, device
63         )
64         val_loss, val_acc = evaluate(model, val_loader, criterion, device)
65         scheduler.step()
66         print(
67             f"epoch={epoch + 1} train_loss={train_loss:.4f} "
68             f"train_acc={train_acc:.4f} val_loss={val_loss:.4f} "
69             f"val_acc={val_acc:.4f}"
70         )
71 if __name__ == "__main__":
72     main()
```

常见坑

三类坑最常见：忘记 `optimizer.zero_grad()` 会让梯度跨 batch 累加；验证时忘记 `model.eval()` 会让 Dropout 和 BatchNorm 仍按训练模式工作；统计 loss 时应累加 `loss.item()`，否则可能保留计算图并导致显存增长。

2.7 面试高频问答

面试题

使用 `CrossEntropyLoss` 要先 softmax 吗？

参考答案

不要。`nn.CrossEntropyLoss` 内部已经包含 `LogSoftmax` 和 `NLLLoss`，输入应是 logits。手动 softmax 会降低数值稳定性，也可能影响梯度。`target` 应是类别索引。

面试题

为什么必须 `zero_grad()`？

参考答案

PyTorch 默认累加梯度，而不是自动覆盖梯度。普通训练中如果不清零，本 batch 梯度会叠加历史 batch 梯度，导致更新方向和步长错误。只有明确做梯度累积时才故意不每步清零。

面试题

`model.train()` 与 `model.eval()` 有什么区别？

参考答案

它们切换模块行为，主要影响 Dropout 和 BatchNorm。训练模式启用 Dropout 并更新 BatchNorm running 统计量；推理模式关闭 Dropout，并使用累计统计量。关闭梯度要靠 `torch.no_grad()`。

面试题

Adam 和 SGD 如何取舍？

参考答案

Adam 收敛快、调参省心，适合快速建立 baseline；SGD 加 momentum 简单稳定，在部分视觉任务中泛化好但更依赖学习率调度。现代项目常优先试 AdamW，再按任务验证 SGD。

面试题

`DataLoader` 的 `num_workers` 是什么？越大越好吗？

参考答案

它控制用多少子进程加载数据，可减少 GPU 等数据的时间。但不是越大越好，过大会增加进程切换、内存占用和 I/O 竞争。调试或 Windows 环境常用 0，训练时逐步测试 2、4、8。

2.8 章末速记

面试速记卡

标准训练循环模板

1. `model.to(device)`, 每个 batch 的 `x` 和 `y` 也要到同一 device。
2. 训练前 `model.train()`。
3. 五步:`optimizer.zero_grad()` → `logits = model(x)` → `loss = criterion(logits, y)` → `loss.backward()` → `optimizer.step()`。
4. 验证前 `model.eval()`, 外层写 `with torch.no_grad():`。
5. `CrossEntropyLoss` 吃 logits, target 是类别索引; 统计 loss 用 `loss.item()`。

Chapter 3

必会网络速查：MLP / CNN / RNN / Transformer

3.1 本章定位

面试问网络结构，重点不是背论文细节，而是讲清楚：结构是什么、输入输出形状如何变化、参数量怎么算、解决了什么问题、有什么常见坑。本章按 MLP、CNN、RNN/LSTM/GRU、Transformer 四类速查，并给出 PyTorch 2.x 最小可跑代码。

面试速记卡

答网络结构题的顺序：输入形状 → 核心层 → 输出形状 → 参数量 → 训练难点 → 为什么有效。

3.2 MLP：全连接网络

3.2.1 结构与直觉

MLP (Multi-Layer Perceptron, 多层感知机) 由线性层和非线性激活函数堆叠而成：

$$x \rightarrow \text{Linear} \rightarrow \text{ReLU} \rightarrow \text{Linear} \rightarrow y.$$

线性层做仿射变换，激活函数提供非线性。若输入维度为 d_{in} ，隐藏层维度为 h ，输出维度为 d_{out} ，两层 MLP 参数量为：

$$d_{in}h + h + hd_{out} + d_{out}.$$

MLP 简单通用，但不显式利用图像局部性，也不显式建模序列顺序。

3.2.2 最小 MLP 代码

```
1 import torch
2 import torch.nn as nn
3
4 model = nn.Sequential(
```



```

5     nn.Linear(10, 32),
6     nn.ReLU(),
7     nn.Linear(32, 2),
8 )
9
10 x = torch.randn(4, 10)
11 y = model(x)
12 print(y.shape)

```

输入是 [batch, features], 输出是 [4, 2]。

3.3 CNN: 局部特征提取器

3.3.1 Conv2d 参数与输出尺寸

`nn.Conv2d` 的常见参数包括: `in_channels`、`out_channels`、`kernel_size`、`stride`、`padding`。输入通常是 [N, C, H, W], 输出通道数等于卷积核个数。

对单个空间方向, 卷积输出尺寸为:

$$H_{out} = \left\lfloor \frac{H_{in} + 2p - k}{s} \right\rfloor + 1.$$

宽度方向同理:

$$W_{out} = \left\lfloor \frac{W_{in} + 2p - k}{s} \right\rfloor + 1.$$

其中 p 是 padding, k 是 kernel size, s 是 stride。

常见坑

“padding 为 1 尺寸不变”只在 $k = 3$ 、 $s = 1$ 时成立。若 $k = 5$ 、 $s = 1$, 要保持尺寸通常需要 $p = 2$ 。

3.3.2 池化、感受野与参数量

池化用于下采样。最大池化保留局部最强响应, 平均池化保留局部均值, 二者通常没有可学习参数。感受野指输出特征图上一个位置能看到的输入区域。堆叠小卷积可以扩大感受野; 两个 3×3 卷积在步幅为 1 时有效感受野相当于 5×5 , 但参数更少且多一次非线性。

普通二维卷积参数量为:

$$k_h k_w C_{in} C_{out} + C_{out}.$$

若 `bias=False`, 则没有最后的 C_{out} 。

3.3.3 1×1 卷积

1×1 卷积不混合空间邻域, 只在通道维度做线性组合。它常用于升维或降维、通道融合、瓶颈结构中减少后续 3×3 卷积的计算量。例如从 256 通道降到 64 通道, 参数量是 $1 \times 1 \times 256 \times 64 + 64$ 。

3.3.4 最小 CNN 代码

```

1 import torch
2 import torch.nn as nn
3
4 model = nn.Sequential(
5     nn.Conv2d(3, 8, kernel_size=3, stride=1, padding=1),
6     nn.ReLU(),
7     nn.MaxPool2d(kernel_size=2),
8     nn.Conv2d(8, 16, kernel_size=3, stride=1, padding=1),
9     nn.ReLU(),
10    nn.AdaptiveAvgPool2d((1, 1)),
11    nn.Flatten(),
12    nn.Linear(16, 10),
13 )
14
15 x = torch.randn(4, 3, 32, 32)
16 y = model(x)
17 print(y.shape)

```

第一层卷积保持 32×32 ，池化后变成 16×16 。`AdaptiveAvgPool2d((1, 1))` 把每个通道压成一个数，因此线性层输入维度是 16。

3.3.5 LeNet 思路

LeNet 的主线是：

Conv $\rightarrow$ Pool $\rightarrow$ Conv $\rightarrow$ Pool $\rightarrow$ MLP.

它体现了 CNN 的经典范式：卷积提局部特征，池化降采样，全连接做分类。现代 CNN 更深更宽，但这个思路仍是基础。

3.3.6 ResNet 残差块

ResNet 让网络学习残差：

$$y = F(x) + x.$$

若最优映射接近恒等映射，令 $F(x)$ 接近 0 比直接学习 $H(x) = x$ 更容易。

```

1 import torch
2 import torch.nn as nn
3
4 class BasicBlock(nn.Module):
5     def __init__(self, channels):
6         super().__init__()
7         self.net = nn.Sequential(
8             nn.Conv2d(channels, channels, 3, padding=1, bias=False),
9             nn.BatchNorm2d(channels),

```

```

10         nn.ReLU(),
11         nn.Conv2d(channels, channels, 3, padding=1, bias=False),
12         nn.BatchNorm2d(channels),
13     )
14     self.relu = nn.ReLU()
15
16     def forward(self, x):
17         out = self.net(x)
18         out = out + x
19         return self.relu(out)
20
21 block = BasicBlock(8)
22 x = torch.randn(2, 8, 16, 16)
23 y = block(x)
24 print(y.shape)

```

残差连接缓解梯度消失的直觉是：反向传播时梯度可以沿 shortcut 更直接地传回浅层，不必完全穿过多层非线性变换。它不是保证梯度永不消失，而是提供更短、更稳定的优化路径。

3.4 RNN / LSTM / GRU: 序列建模

3.4.1 RNN 与梯度消失

RNN 用隐藏状态递归处理序列：

$$h_t = f(x_t, h_{t-1}).$$

它适合文本、时间序列等有顺序的数据。普通 RNN 的问题是长序列上容易梯度消失或爆炸，因为反向传播要反复乘以权重矩阵和激活函数导数，小于 1 的因子连乘会迅速变小。

3.4.2 LSTM 的三个门

LSTM 引入 cell state c_t ，让信息更稳定地跨时间传播。三个门的直觉是：遗忘门决定旧记忆丢掉什么，输入门决定新信息写入多少，输出门决定从 cell state 暴露多少到隐藏状态 h_t 。可以把 cell state 理解成信息高速公路，门控机制负责选择性保留、写入和输出。

3.4.3 GRU 简化

GRU 是 LSTM 的简化版本，没有单独的 cell state，通常用更新门和重置门控制信息流。它参数更少、计算更轻，在一些短序列或小数据任务上表现不输 LSTM。

3.4.4 nn.LSTM 最小用法

```

1 import torch
2 import torch.nn as nn
3

```

```

4  lstm = nn.LSTM(
5      input_size=5,
6      hidden_size=8,
7      num_layers=1,
8      batch_first=True,
9  )
10
11  x = torch.randn(4, 6, 5)
12  out, (h_n, c_n) = lstm(x)
13
14  print(out.shape)
15  print(h_n.shape)
16  print(c_n.shape)

```

当 `batch_first=True` 时, 输入形状是 `[batch, seq, feature]`。out 形状是 `[4, 6, 8]`, 包含每个时间步输出; `h_n` 和 `c_n` 形状是 `[1, 4, 8]`。

3.5 Transformer: 注意力与并行序列建模

3.5.1 Self-Attention 公式

给定查询 Q 、键 K 、值 V , 缩放点积注意力为:

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^\top}{\sqrt{d_k}}\right)V.$$

它让每个位置都能直接关注序列中其他位置。

3.5.2 为何除以 $\sqrt{d_k}$

若 Q 和 K 的元素方差大致为 1, 点积 $QK^\top$ 的尺度会随 d_k 增大。不缩放时 softmax 输入可能过大, 分布过尖, 梯度变小。除以 $\sqrt{d_k}$ 可以稳定数值尺度和训练过程。

3.5.3 Multi-Head、位置编码与结构

Multi-head attention 把表示拆到多个子空间, 让不同头学习不同依赖关系, 再拼接投影融合。Self-attention 本身对顺序不敏感, 因此需要位置编码注入位置信息, 常见做法有正弦余弦位置编码、可学习位置向量和相对位置编码。

Encoder 通常堆叠 self-attention 和 feed-forward 子层, 并配残差连接和 LayerNorm。Decoder 在 masked self-attention 后加入 cross-attention, 用来读取 encoder 输出; mask 防止当前位置看到未来 token。

3.5.4 相比 RNN 的优势

RNN 的 h_t 依赖 h_{t-1} , 训练时难以完全并行。Transformer 的 self-attention 可一次性计算所有位置之间的相关性, 因此并行性更强; 任意两个位置之间只需一次 attention 连接, 长依赖路径更短。代价是标准 attention 对序列长度 n 的时间和显存复杂度为 $O(n^2)$ 。

3.5.5 nn.MultiheadAttention 最小示例

```
1 import torch
2 import torch.nn as nn
3
4 attn = nn.MultiheadAttention(
5     embed_dim=16,
6     num_heads=4,
7     batch_first=True,
8 )
9
10 x = torch.randn(2, 5, 16)
11 y, weights = attn(x, x, x)
12
13 print(y.shape)
14 print(weights.shape)
```

这里 $Q = K = V = x$ ，所以是 self-attention。输出 y 形状是 $[2, 5, 16]$ ；默认 $weights$ 是平均后的注意力权重，通常为 $[batch, target_len, source_len]$ 。

3.6 面试题速答

面试题

卷积输出尺寸怎么算？

参考答案

单个方向使用 $H_{out} = \lfloor (H_{in} + 2p - k) / s \rfloor + 1$ 。其中 p 是 padding, k 是 kernel size, s 是 stride。宽度方向同理。

面试题

1×1 卷积干嘛？

参考答案

它在通道维度做线性组合，不扩大空间感受野。常用于升降维、通道融合、瓶颈结构降计算量。

面试题

残差为何有效？

参考答案

残差块输出 $F(x) + x$, 让网络学习相对输入的增量。shortcut 给梯度提供更短路径, 深层网络更容易优化。

面试题

LSTM 为何缓解梯度消失?

参考答案

LSTM 用 cell state 保存长期信息, 并用遗忘门、输入门、输出门控制信息流, 减少普通 RNN 中长链式连乘带来的梯度衰减。

面试题

Attention 为何 scale?

参考答案

点积尺度会随 d_k 增大。不除以 $\sqrt{d_k}$ 时 softmax 可能过尖, 梯度变小; 缩放能稳定训练。

面试题

Multi-head 的意义是什么?

参考答案

多个头能在不同子空间学习不同关系, 例如局部依赖、长距离依赖或语义关系, 最后拼接投影融合。

面试题

Transformer 为何能并行?

参考答案

RNN 要按时间步递推; Transformer 的 self-attention 可同时计算所有位置之间的相关性, 所以训练阶段并行性更强。

3.7 章末速记卡

面试速记卡		
网络	一句话要点	关键公式或关键词
MLP	全连接加非线性，通用但结构先验少	$Wx + b$
CNN	局部连接和权值共享，适合图像	$H_{out} = \lfloor (H_{in} + 2p - k)/s \rfloor + 1$
RNN	递归处理序列，长依赖训练困难	$h_t = f(x_t, h_{t-1})$
LSTM	门控加 cell state 保留长期信息	遗忘门、输入门、输出门
GRU	LSTM 的轻量简化版本	更新门、重置门
Transformer	attention 建模全局依赖且可并行	$\text{softmax}(QK^\top/\sqrt{d_k})V$

面试速记卡	
一句话记忆：MLP 做通用函数拟合；CNN 抓局部空间特征；RNN/LSTM/GRU 按时间递推建模序列；Transformer 用 attention 直接建模全局依赖。	

Chapter 4

高频理论面试题速记

4.1 反向传播与梯度直觉

面试题

反向传播的本质是什么？

参考答案

反向传播就是在计算图上从后往前应用链式法则。前向传播计算输出并保存中间量；反向传播把损失对输出的梯度逐层传回，乘上每个节点的局部导数，得到所有参数的梯度。它负责“算梯度”，优化器才负责“更新参数”。

面试题

链式法则在神经网络中如何体现？

参考答案

若 L 依赖 y ， y 依赖 x ，则

$$\frac{\partial L}{\partial x} = \frac{\partial L}{\partial y} \frac{\partial y}{\partial x}.$$

深层网络是函数复合，反传时梯度沿路径把各层局部梯度连乘。反向传播的高效之处在于复用中间梯度，避免对每个参数单独做数值求导。

面试题

手推两层全连接网络的梯度直觉。

参考答案

设 $z_1 = W_1x + b_1$ ， $h = \phi(z_1)$ ， $z_2 = W_2h + b_2$ ，损失为 L 。输出层误差记为 $\delta_2 = \partial L / \partial z_2$ ，则

$$\frac{\partial L}{\partial W_2} = \delta_2 h^\top, \quad \frac{\partial L}{\partial b_2} = \delta_2.$$

隐藏层误差为 $\delta_1 = (W_2^\top \delta_2) \odot \phi'(z_1)$ ，所以

$$\frac{\partial L}{\partial W_1} = \delta_1 x^\top, \quad \frac{\partial L}{\partial b_1} = \delta_1.$$

一句话：参数梯度约等于“本层误差信号”乘“本层输入”。

常见坑

常见误区：把反向传播说成“从输出层开始更新参数”。准确说法是先反向计算梯度，再由 SGD、Adam 等优化器根据梯度更新参数。

4.2 梯度消失与梯度爆炸

面试题

梯度消失和梯度爆炸的成因是什么？

参考答案

核心原因是反传中的连乘。若每层导数或雅可比矩阵范数长期小于 1，梯度会越来越小，形成梯度消失；若长期大于 1，梯度会急剧变大，形成梯度爆炸。sigmoid、tanh 在饱和区导数接近 0，会进一步加剧梯度消失。

面试题

训练中如何识别梯度消失或爆炸？

参考答案

梯度消失常表现为浅层几乎学不动、loss 下降很慢、深层网络退化；RNN 中表现为难以学习长距离依赖。梯度爆炸常表现为 loss 剧烈震荡、参数范数异常增大、梯度范数很大，甚至出现 NaN。

面试题

如何缓解梯度消失和梯度爆炸？

参考答案

常见方法：使用 ReLU、LeakyReLU、GELU 等非饱和激活；用 Xavier 或 He 初始化保持方差稳定；加入 BatchNorm 或 LayerNorm；使用残差连接缩短梯度路径；对爆炸使用梯度裁剪；序列模型用 LSTM、GRU 的门控结构缓解长依赖训练困难。

4.3 激活函数对比

面试题

为什么神经网络必须使用激活函数？

参考答案

激活函数提供非线性。若没有激活函数，多层线性层复合仍等价于一个线性变换，网络再深也只能表达线性函数。加入非线性后，网络才能拟合复杂边界和高维非线性映射。

面试题

sigmoid、tanh、ReLU、LeakyReLU、GELU 如何对比？

参考答案

面试可按输出范围、优点、缺点和适用场景回答：

激活	输出范围	优点	缺点与适用
sigmoid	$(0, 1)$	概率解释	易饱和、非零中心；二分类输出或门控
tanh	$(-1, 1)$	零中心	仍易饱和；早期 RNN 常见
ReLU	$[0, \infty)$	简单、收敛快	可能死亡 ReLU；CNN/MLP 常用
LeakyReLU	$(-\infty, \infty)$	负半轴有梯度	斜率需设定；缓解死亡 ReLU
GELU	$(-\infty, \infty)$	平滑、效果好	计算稍复杂；Transformer 常用

面试题

什么是死亡 ReLU？为什么 GELU 常用于 Transformer？

参考答案

死亡 ReLU 指神经元输入长期小于 0，输出和梯度都为 0，导致该神经元不再更新，可用较小学习率、合适初始化或 LeakyReLU 缓解。GELU 近似 $x\Phi(x)$ ，是平滑的软门控，负半轴也保留小幅信息，经验上适合 Transformer 的前馈层。

4.4 BatchNorm 与 LayerNorm

面试题

BatchNorm 的原理是什么？

参考答案

BatchNorm 对 mini-batch 内每个通道做标准化, 再用可学习参数恢复表达能力:

$$\mu_B = \frac{1}{m} \sum_i x_i, \quad \sigma_B^2 = \frac{1}{m} \sum_i (x_i - \mu_B)^2,$$

$$\hat{x}_i = \frac{x_i - \mu_B}{\sqrt{\sigma_B^2 + \epsilon}}, \quad y_i = \gamma \hat{x}_i + \beta.$$

它让中间特征分布更稳定, 通常能加快收敛并提升训练稳定性。

面试题

BatchNorm 训练和推理时有什么区别?

参考答案

训练时使用当前 mini-batch 的均值和方差, 并更新 running mean 与 running var。推理时不再用当前 batch 统计, 而使用训练阶段累计的 running 统计量, 因此输出不依赖当前 batch 的样本组成。PyTorch 中要通过 `model.train()` 和 `model.eval()` 切换行为。

面试题

BatchNorm 为什么有效? 放在激活前还是后?

参考答案

BN 有效主要因为它稳定中间层尺度, 改善优化条件, 使网络能使用更大学习率, 并带来轻微 batch 噪声正则化。经典 CNN 中最常见顺序是 Conv -> BN -> ReLU, 即卷积后、激活前; 也有激活后 BN 的设计, 实际以架构和实验为准。

面试题

LayerNorm 和 BatchNorm 的区别是什么?

参考答案

BatchNorm 沿 batch 维度统计, 通常对每个通道归一化, 依赖 batch size, 适合 CNN 和较大 batch。LayerNorm 在单个样本内部沿特征维度统计, 不依赖 batch, 训练和推理行为一致, 因此更适合 NLP、RNN 和 Transformer。

面试题

为什么 Transformer 常用 LayerNorm?

参考答案

Transformer 常面对变长序列、小 batch、分布变化和自回归推理。BatchNorm 的 batch 统计可能不稳定，且训练推理行为不同；LayerNorm 对每个 token 或样本内部特征归一化，不依赖其他样本，因此更自然、更稳定。

4.5 Dropout、过拟合与欠拟合

面试题

Dropout 的原理是什么？

参考答案

Dropout 在训练时以概率 p 随机把部分神经元输出置零，迫使模型不要过度依赖少数特征，可看作训练许多子网络并做隐式集成。它主要用于缓解过拟合，常见于全连接层和 Transformer 的若干位置。

面试题

训练和推理时 Dropout 有什么区别？什么是 inverted dropout？

参考答案

训练时随机丢弃，推理时关闭丢弃。PyTorch 默认使用 inverted dropout：若 $m \sim \text{Bernoulli}(1-p)$ ，训练输出为

$$y = \frac{m \odot x}{1-p}.$$

因此 $\mathbb{E}[y] = x$ ，推理时无需再乘保留概率。验证时忘记 `eval()` 会让指标随机波动。

面试题

如何诊断过拟合和欠拟合？

参考答案

看 train/val 曲线。过拟合：训练 loss 低、训练指标好，但验证 loss 上升或验证指标变差。欠拟合：训练和验证表现都差，训练 loss 也降不下来。先判断问题类型，再决定是加强正则还是提升模型能力。

面试题

过拟合和欠拟合分别怎么处理？

参考答案

过拟合可用 L2/weight decay、Dropout、数据增强、early stopping、更多数据、标签平滑、减小模型容量。欠拟合可增大模型、训练更久、减弱正则、改进特征或结构、调整学习率和优化器。口诀：过拟合管住模型，欠拟合放开模型。

4.6 权重初始化

面试题

为什么权重初始化很重要？

参考答案

初始化影响前向激活和反向梯度的方差。权重太小，信号逐层衰减；权重太大，激活和梯度可能爆炸。好的初始化目标是让各层输入、输出和梯度处在合适尺度，使训练一开始就比较稳定。

面试题

Xavier 和 He 初始化分别适合什么场景？

参考答案

Xavier 适合 sigmoid、tanh 等较对称激活，典型方差为

$$\text{Var}(W) = \frac{2}{\text{fan}_{\text{in}} + \text{fan}_{\text{out}}}.$$

He 初始化适合 ReLU/LeakyReLU，因为 ReLU 会截断约一半激活，常用

$$\text{Var}(W) = \frac{2}{\text{fan}_{\text{in}}}.$$

面试中可答：tanh 用 Xavier，ReLU 用 He。

4.7 优化器

面试题

SGD 和 Momentum 的更新式与特点是什么？

参考答案

SGD 使用小批量梯度 $g_t = \nabla_{\theta} L(\theta_t)$ ：

$$\theta_{t+1} = \theta_t - \eta g_t.$$

它简单、显存省、泛化常好，但收敛慢且对学习率敏感。Momentum 引入速度

$$v_t = \mu v_{t-1} + g_t, \quad \theta_{t+1} = \theta_t - \eta v_t.$$

它在稳定方向加速，在震荡方向抵消，通常比普通 SGD 更稳。

面试题

Adam 的更新公式是什么？

参考答案

Adam 同时估计一阶矩和二阶矩：

$$\begin{aligned}g_t &= \nabla_{\theta} L(\theta_t), \quad m_t = \beta_1 m_{t-1} + (1 - \beta_1) g_t, \\v_t &= \beta_2 v_{t-1} + (1 - \beta_2) g_t^2, \quad \hat{m}_t = \frac{m_t}{1 - \beta_1^t}, \quad \hat{v}_t = \frac{v_t}{1 - \beta_2^t}, \\ \theta_{t+1} &= \theta_t - \eta \frac{\hat{m}_t}{\sqrt{\hat{v}_t} + \epsilon}.\end{aligned}$$

m_t 类似动量， v_t 用于按参数自适应缩放学习率。

面试题

Adam、AdamW、SGD Momentum 如何取舍？

参考答案

Adam 收敛快、调参相对容易，适合稀疏梯度、NLP 和 Transformer。AdamW 将权重衰减与 Adam 梯度更新解耦，weight decay 含义更纯粹，是现代 Transformer 的常用默认选择。SGD Momentum 在 CNN 分类等任务上最终泛化可能很好，但通常需要更仔细调学习率。

4.8 学习率

面试题

学习率过大或过小分别有什么现象？

参考答案

学习率过大时，loss 震荡、不收敛，甚至参数爆炸和 NaN；过小时，loss 下降很慢，训练时间长，可能停在较差区域。学习率通常是最重要的超参数，要和 batch size、优化器、归一化方式一起考虑。

面试题

什么是 warmup？为什么常配合 AdamW 和 Transformer？

参考答案

warmup 指训练初期从很小学习率逐步升到目标学习率。模型刚开始参数、梯度和归一化统计不稳定，直接用大学习率容易发散；warmup 能平滑启动训练。Transformer、大 batch、AdamW 训练中很常见。

面试题

常见学习率衰减策略有哪些？

参考答案

常见策略包括 step decay、multi-step decay、exponential decay、cosine annealing、linear decay、ReduceLROnPlateau。直觉是前期较大学习率快速搜索，后期降低学习率精细收敛；现代训练常用 warmup 加 cosine 或 linear decay。

4.9 损失函数

面试题

MSE 和交叉熵分别适合什么任务？

参考答案

MSE 为

$$L_{\text{MSE}} = \frac{1}{n} \sum_{i=1}^n (\hat{y}_i - y_i)^2,$$

常用于回归，并有高斯噪声下的最大似然解释。多分类交叉熵为

$$L_{\text{CE}} = - \sum_{k=1}^K y_k \log p_k, \quad p_k = \frac{e^{z_k}}{\sum_j e^{z_j}},$$

常用于分类，等价于最大化真实类别概率。

面试题

为什么分类通常用交叉熵而不是 MSE？

参考答案

交叉熵直接匹配概率建模和最大似然目标。softmax 加交叉熵有简洁梯度

$$\frac{\partial L}{\partial z_k} = p_k - y_k,$$

模型自信但错误时仍有强纠正信号；MSE 搭配饱和激活时梯度可能变弱，优化效率通常不如交叉熵。

面试题

PyTorch 中为什么常把 logits 直接传给损失函数？

参考答案

CrossEntropyLoss 期望输入未归一化 logits，内部实现 `log_softmax` 加 `NLLLoss`，比先手动 `softmax` 再取 `log` 更数值稳定。二分类同理，优先用 `BCEWithLogitsLoss`，它把 `sigmoid` 和 `BCE` 合并，避免数值下溢或上溢。

4.10 综合追问**面试题**

训练 loss 不下降，你会如何排查？

参考答案

先查数据、标签和输入归一化，再查学习率是否过大或过小；确认模型处于 `train()`，梯度没有被 `detach`，也没有忘记 `zero_grad()`。再检查输出和损失是否匹配，例如分类是否把 logits 传给 `CrossEntropyLoss`。最后考虑初始化、归一化、优化器和模型容量。

面试题

BatchNorm、Dropout、Weight Decay 都能防过拟合吗？

参考答案

它们都可能改善泛化，但机制不同。Weight Decay 直接惩罚大权重；Dropout 随机屏蔽神经元，减少共适应；BatchNorm 主要稳定优化，同时 batch 噪声带来一定正则化效果。不要把 BN 只理解成正则化方法。

面试题

残差连接为什么能缓解深层网络训练困难？

参考答案

残差块学习 $F(x) + x$ ，恒等分支给前向信号和反向梯度提供捷径。反传时梯度可以绕过若干非线性层直接传到浅层，减轻梯度消失，使很深的网络更容易优化。

面试速记卡**第 4 章面试速答清单**

- 反向传播：计算图上的链式法则；每层梯度约等于误差信号乘本层输入。
- 梯度问题：连乘导致消失/爆炸；用激活、初始化、归一化、残差、裁剪和 LSTM/GRU

缓解。

- 激活函数: sigmoid/tanh 易饱和; ReLU 快但可能死亡; LeakyReLU 保留负梯度; GELU 平滑, Transformer 常用。
- BatchNorm: 训练用 batch 统计并更新 running 统计; 推理用 running mean/var; CNN 常见 Conv $\rightarrow$ BN $\rightarrow$ ReLU。
- LayerNorm: 对单样本特征归一化, 不依赖 batch, 训练推理一致, 所以 NLP/Transformer 常用。
- Dropout: 训练随机置零, 推理关闭; inverted dropout 训练时除以 $1 - p$ 保持期望。
- 过拟合: train 好、val 差; 用 L2、Dropout、数据增强、early stopping、更多数据或减小模型。
- 欠拟合: train 和 val 都差; 增大模型、训练更久、减弱正则、改结构或调优化。
- 初始化: tanh/sigmoid 用 Xavier; ReLU/LeakyReLU 用 He; 目标是稳定方差。
- 优化器: SGD 泛化常好但慢; Momentum 加速; Adam 自适应; AdamW 解耦 weight decay。
- 学习率: 过大震荡或 NaN, 过小收敛慢; warmup 稳定启动, 衰减帮助后期收敛。
- 损失: 回归用 MSE; 分类用交叉熵, 因其匹配概率最大似然且 softmax 梯度为 $p - y$ 。

Chapter 5

高频编码面试题

5.1 手写 Scaled Dot-Product Attention

面试题

给定 q 、 k 、 v ，手写缩放点积注意力。要求支持 $mask$ ， $mask$ 中 **False** 的位置不能被关注。

参考答案

公式为

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^T}{\sqrt{d_k}}\right)V.$$

```
1 import math
2 import torch
3
4
5 def scaled_dot_product_attention(q, k, v, mask=None, dropout=None):
6     """
7     q: (... , tgt_len, d_k)
8     k: (... , src_len, d_k)
9     v: (... , src_len, d_v)
10    mask: broadcastable to (... , tgt_len, src_len), True means keep
11    """
12    scores = torch.matmul(q, k.transpose(-2, -1))
13    scores = scores / math.sqrt(q.size(-1))
14    if mask is not None:
15        scores = scores.masked_fill(~mask, torch.finfo(scores.dtype).min)
16    attn = torch.softmax(scores, dim=-1)
17    if dropout is not None:
18        attn = dropout(attn)
19    out = torch.matmul(attn, v)
20    return out, attn
```

要点 `k.transpose(-2, -1)` 只交换最后两维；`mask` 要能广播到注意力分数形状；`softmax` 沿 `key` 维，即 `dim=-1`。

常见坑

不要先 `softmax` 再 `mask`；被屏蔽位置应在 `softmax` 前填成足够小的值。

5.2 手写 Multi-Head Attention

面试题

用 `nn.Linear` 实现多头注意力：线性投影、拆头、缩放点积注意力、合头、输出投影。

参考答案

多头注意力把 `embed_dim` 分成多个 `head_dim`，每个头独立做 attention。

```
1 import math
2 import torch
3 from torch import nn
4
5
6 class MultiHeadAttention(nn.Module):
7     def __init__(self, embed_dim, num_heads, dropout=0.0):
8         super().__init__()
9         if embed_dim % num_heads != 0:
10             raise ValueError("embed_dim must be divisible by num_heads")
11         self.embed_dim = embed_dim
12         self.num_heads = num_heads
13         self.head_dim = embed_dim // num_heads
14         self.q_proj = nn.Linear(embed_dim, embed_dim)
15         self.k_proj = nn.Linear(embed_dim, embed_dim)
16         self.v_proj = nn.Linear(embed_dim, embed_dim)
17         self.out_proj = nn.Linear(embed_dim, embed_dim)
18         self.dropout = nn.Dropout(dropout)
19
20     def split_heads(self, x):
21         bsz, seq_len, _ = x.shape
22         x = x.view(bsz, seq_len, self.num_heads, self.head_dim)
23         return x.transpose(1, 2)
24
25     def merge_heads(self, x):
26         bsz, heads, seq_len, head_dim = x.shape
27         x = x.transpose(1, 2).contiguous()
28         return x.view(bsz, seq_len, heads * head_dim)
```

```

29
30     def forward(self, query, key, value, mask=None):
31         q = self.split_heads(self.q_proj(query))
32         k = self.split_heads(self.k_proj(key))
33         v = self.split_heads(self.v_proj(value))
34         scores = q @ k.transpose(-2, -1) / math.sqrt(self.head_dim)
35         if mask is not None:
36             if mask.dim() == 3:
37                 mask = mask.unsqueeze(1)
38                 scores = scores.masked_fill(~mask, torch.finfo(scores.dtype).min)
39         attn = self.dropout(torch.softmax(scores, dim=-1))
40         out = self.merge_heads(attn @ v)
41         return self.out_proj(out), attn

```

要点 输入常用 (batch, seq_len, embed_dim); 拆头后是 (batch, heads, seq_len, head_dim); transpose 后再 view 前通常要 contiguous()。

5.3 用 nn.Parameter 手写 Linear 层

面试题

不用 nn.Linear, 只用 nn.Parameter 手写全连接层, 包含初始化和 forward。

参考答案

线性层计算 $Y = XW^T + b$ 。PyTorch 线性层权重形状为 (out_features, in_features)。

```

1  import math
2  import torch
3  from torch import nn
4
5
6  class MyLinear(nn.Module):
7      def __init__(self, in_features, out_features, bias=True):
8          super().__init__()
9          self.in_features = in_features
10         self.out_features = out_features
11         self.weight = nn.Parameter(torch.empty(out_features, in_features))
12         if bias:
13             self.bias = nn.Parameter(torch.empty(out_features))
14         else:
15             self.register_parameter("bias", None)
16         self.reset_parameters()
17

```

```

18     def reset_parameters(self):
19         bound = 1.0 / math.sqrt(self.in_features)
20         nn.init.uniform_(self.weight, -bound, bound)
21         if self.bias is not None:
22             nn.init.uniform_(self.bias, -bound, bound)
23
24     def forward(self, x):
25         y = x.matmul(self.weight.t())
26         if self.bias is not None:
27             y = y + self.bias
28         return y

```

要点 可训练张量必须是 `nn.Parameter`; 无 bias 时用 `register_parameter("bias", None)`; bias 自动广播到最后一维。

5.4 手写 BatchNorm1d

面试题

手写 BatchNorm1d, 区分训练/推理分支, 训练时更新 `running_mean` 与 `running_var`。

参考答案

BatchNorm 先归一化再仿射变换: $\hat{x} = (x - \mu) / \sqrt{\sigma^2 + \epsilon}$, $y = \gamma \hat{x} + \beta$ 。

```

1  import torch
2  from torch import nn
3
4
5  class MyBatchNorm1d(nn.Module):
6      def __init__(self, num_features, eps=1e-5, momentum=0.1, affine=True):
7          super().__init__()
8          self.eps = eps
9          self.momentum = momentum
10         self.affine = affine
11         if affine:
12             self.weight = nn.Parameter(torch.ones(num_features))
13             self.bias = nn.Parameter(torch.zeros(num_features))
14         else:
15             self.register_parameter("weight", None)
16             self.register_parameter("bias", None)
17         self.register_buffer("running_mean", torch.zeros(num_features))
18         self.register_buffer("running_var", torch.ones(num_features))
19

```

```

20     def forward(self, x):
21         if x.dim() not in (2, 3):
22             raise ValueError("expected 2D or 3D input")
23         reduce_dims = (0,) if x.dim() == 2 else (0, 2)
24         shape = (1, -1) if x.dim() == 2 else (1, -1, 1)
25         if self.training:
26             mean = x.mean(dim=reduce_dims, keepdim=True)
27             var = x.var(dim=reduce_dims, unbiased=False, keepdim=True)
28             with torch.no_grad():
29                 self.running_mean.mul_(1 - self.momentum).add_(
30                     self.momentum * mean.view(-1)
31                 )
32                 self.running_var.mul_(1 - self.momentum).add_(
33                     self.momentum * var.view(-1)
34                 )
35         else:
36             mean = self.running_mean.view(shape)
37             var = self.running_var.view(shape)
38             y = (x - mean) / torch.sqrt(var + self.eps)
39             if self.affine:
40                 y = y * self.weight.view(shape) + self.bias.view(shape)
41         return y

```

要点 running stats 是 buffer 不是参数；(N,C,L) 要在 N 和 L 维统计；更新 buffer 放进 `torch.no_grad()`。

常见坑

官方实现对 running variance 的无偏估计更细；面试重点通常是分支、buffer 与广播。

5.5 手写 Dropout

面试题

实现 inverted dropout：训练时随机置零并缩放，推理时直接返回输入。

参考答案

inverted dropout 在训练阶段除以 1-p，推理阶段无需缩放。

```

1  import torch
2  from torch import nn
3
4

```

```
5 class MyDropout(nn.Module):
6     def __init__(self, p=0.5):
7         super().__init__()
8         if p < 0.0 or p >= 1.0:
9             raise ValueError("p must be in [0, 1)")
10        self.p = p
11
12    def forward(self, x):
13        if not self.training or self.p == 0.0:
14            return x
15        keep_prob = 1.0 - self.p
16        mask = torch.rand_like(x) < keep_prob
17        return x * mask.to(x.dtype) / keep_prob
```

要点 训练时输出期望保持不变；推理时不要再乘 $1-p$ ；用 `torch.rand_like(x)` 可自动匹配 device。

5.6 手写交叉熵损失

面试题

手写多分类交叉熵损失，要求用 `log_softmax + nll`，输入是 logits 和类别索引。

参考答案

不要先 softmax 再取 log；稳定写法是直接计算 `log_softmax`。

```
1 import torch
2 import torch.nn.functional as F
3
4
5 def cross_entropy_from_logits(logits, target, reduction="mean"):
6     log_probs = F.log_softmax(logits, dim=-1)
7     nll = -log_probs.gather(1, target.view(-1, 1)).squeeze(1)
8     if reduction == "mean":
9         return nll.mean()
10    if reduction == "sum":
11        return nll.sum()
12    if reduction == "none":
13        return nll
14    raise ValueError("invalid reduction")
```

要点 logits 不需要预先归一化；target 形状通常是 (batch,) 且 dtype 为 `torch.long`；gather 的 index 维度要匹配。

5.7 手写完整训练循环

面试题

实现 `train_one_epoch(model, loader, opt, loss_fn, device)`，包含训练模式、搬运数据、前向、反向、更新和平均损失统计。

参考答案

训练循环的标准顺序是清梯度、前向、算损失、反向、更新参数。

```
1 import torch
2
3
4 def train_one_epoch(model, loader, opt, loss_fn, device):
5     model.train()
6     total_loss = 0.0
7     total_count = 0
8     for batch in loader:
9         if isinstance(batch, (tuple, list)):
10             x, y = batch[0], batch[1]
11         else:
12             x, y = batch["x"], batch["y"]
13         x = x.to(device)
14         y = y.to(device)
15         opt.zero_grad(set_to_none=True)
16         pred = model(x)
17         loss = loss_fn(pred, y)
18         loss.backward()
19         opt.step()
20         batch_size = x.size(0)
21         total_loss += loss.detach().item() * batch_size
22         total_count += batch_size
23     return total_loss / max(total_count, 1)
```

要点 顺序是 `zero_grad` -> `forward` -> `loss` -> `backward` -> `step`；统计 `loss` 时用 `detach().item()`；若 `loss` 是 batch mean，累计时乘 batch size。

常见坑

评估循环才使用 `model.eval()` 和 `torch.no_grad()`；训练循环不能包 `no_grad`。

5.8 张量操作题：不用 Python for 循环

面试题

不用 Python for 循环实现：按行 softmax、按标签取概率、广播计算成对欧氏距离、topk、masked_fill。

参考答案

这类题考维度意识，应优先用广播、gather、topk、masked_fill 等原语。

```
1 import torch
2 import torch.nn.functional as F
3
4
5 def row_softmax(x):
6     return torch.softmax(x, dim=1)
7
8
9 def take_class_prob(probs, labels):
10    return probs.gather(1, labels.view(-1, 1)).squeeze(1)
11
12
13 def pairwise_l2_distance(x, y):
14     diff = x[:, None, :] - y[None, :, :]
15     return torch.sqrt((diff * diff).sum(dim=-1))
16
17
18 def pairwise_l2_distance_fast(x, y):
19     x2 = (x * x).sum(dim=1, keepdim=True)
20     y2 = (y * y).sum(dim=1).unsqueeze(0)
21     dist2 = (x2 + y2 - 2.0 * x @ y.t()).clamp_min(0.0)
22     return torch.sqrt(dist2)
23
24
25 def topk_values_indices(x, k):
26     return torch.topk(x, k=k, dim=1)
27
28
29 def mask_logits(logits, valid_mask):
30     return logits.masked_fill(~valid_mask, torch.finfo(logits.dtype).min)
31
32
33 def demo_tensor_ops():
34     logits = torch.tensor([[1.0, 2.0, 3.0], [3.0, 1.0, 0.0]])
35     labels = torch.tensor([2, 0])
```

```

36     probs = F.softmax(logits, dim=1)
37     chosen = take_class_prob(probs, labels)
38     x = torch.randn(4, 5)
39     y = torch.randn(6, 5)
40     dist = pairwise_l2_distance_fast(x, y)
41     values, indices = topk_values_indices(logits, k=2)
42     mask = torch.tensor([[True, True, False], [True, False, True]])
43     masked = mask_logits(logits, mask)
44     return chosen, dist, values, indices, masked

```

要点 gather 适合按样本取类别；`x[:, None, :]` 和 `y[None, :, :]` 可广播；topk 返回 values 和 indices；先说明 mask 中 `True` 的含义。

5.9 可选：Label Smoothing 交叉熵

面试题

实现 label smoothing 版本交叉熵。输入 logits 和类别索引 target，平滑系数为 smoothing。

参考答案

label smoothing 把少量概率质量分给非正确类别，可缓解模型过度自信。

```

1  import torch
2  import torch.nn.functional as F
3
4  def label_smoothing_cross_entropy(logits, target, smoothing=0.1):
5      num_classes = logits.size(-1)
6      log_probs = F.log_softmax(logits, dim=-1)
7      with torch.no_grad():
8          true_dist = torch.full_like(log_probs, smoothing / (num_classes - 1))
9          true_dist.scatter_(1, target.view(-1, 1), 1.0 - smoothing)
10     loss = -(true_dist * log_probs).sum(dim=-1)
11     return loss.mean()

```

要点 scatter_ 写入正确类别概率；构造软标签不需要梯度；实际任务应保证 num_classes > 1。

面试速记卡

手写模板速记

- Attention/MHA: $QK^T/\sqrt{d_k}$, mask 后对最后一维 softmax；多头要线性投影、拆头、合头。

- Linear/BatchNorm/Dropout: 参数用 `nn.Parameter`, running stats 用 `buffer`, inverted dropout 训练除以 `1-p`。
- CrossEntropy/Train loop: 稳定写法是 `log_softmax + gather`; 训练顺序是清梯度、前向、反向、更新。
- Tensor ops: 多用广播、`gather`、`topk`、`masked_fill`, 并先说明 `mask` 中 `True` 的语义。

Chapter 6

分布式训练面试速通

6.1 为什么需要分布式训练

LLM 时代的训练瓶颈通常不是“代码能不能跑起来”，而是数据规模、模型规模和显存上限同时增长。单卡训练在小模型上足够直接，但当模型参数、激活值、优化器状态和 batch size 都变大时，很快会遇到显存墙和训练时间墙。

6.1.1 三个核心动机

- **数据量太大**：单卡吞吐有限，训练一个 epoch 可能需要数天甚至数周。
- **模型太大**：参数、梯度、优化器状态和激活值无法同时放入单张 GPU。
- **显存墙明显**：以 Adam 为例，参数之外还要保存梯度、一阶动量和二阶动量，训练显存远大于推理显存。

一个常见估算是：仅参数本身， N 个参数在 fp16/bf16 下约占 $2N$ 字节；训练时再加梯度、优化器状态和激活值，实际显存开销会成倍增加。分布式训练的本质，就是把计算、数据或状态拆到多张 GPU 上。

面试速记卡

分布式训练解决两类问题：用更多 GPU 加速吞吐，用更多显存容纳更大的模型。

6.2 DP vs DDP

PyTorch 中常被问到的两个概念是 DataParallel (DP) 和 DistributedDataParallel (DDP)。面试中要明确：DP 是早期易用方案，DDP 是生产训练的主流方案。

6.2.1 DataParallel 的问题

DP 是单进程多线程模式。模型通常放在主 GPU 上，每次前向时复制到其他 GPU，输入被切分后并行计算，最后把结果和梯度汇总回主 GPU。它的问题包括：

- 单进程多线程，Python 层会受到 GIL 影响。

- 主 GPU 承担 gather 和参数更新，负载不均。
- 每次前向复制模型，额外开销明显。
- 扩展性较差，不适合多机训练。

6.2.2 DDP 的优势

DDP 是多进程模式，通常每张 GPU 一个进程。每个进程持有一份模型副本，处理自己负责的数据切片。反向传播时，DDP 在梯度 ready 后触发通信，对梯度做 all-reduce，使各个进程得到一致的平均梯度。

对比项	DataParallel	DistributedDataParallel
进程模型	单进程多线程	多进程，通常每 GPU 一进程
性能	受 GIL 和主卡瓶颈影响	通信计算可重叠，性能更好
负载均衡	主 GPU 压力更大	各 GPU 更均衡
多机支持	不适合	原生支持
推荐程度	不推荐用于正式训练	PyTorch 数据并行首选
典型启动	普通 python 脚本	torchrun 启动

常见坑

面试中不要说 DP 和 DDP 只是写法不同。关键差异是单进程多线程与多进程通信模型，DDP 的扩展性和性能都明显更好。

6.3 DDP 原理与最小代码骨架

6.3.1 基本 workflow

DDP 的核心逻辑可以概括为四步：

1. 每个进程绑定一张 GPU，并初始化进程组。
2. 每个进程构造同样的模型，并用 DDP 包装。
3. 使用 DistributedSampler 切分数据，避免不同进程读到完全相同的数据。
4. 反向传播时对梯度做 all-reduce，保证各模型副本同步更新。

DDP 中每个进程都有完整模型副本，所以它主要解决数据并行吞吐问题，并不从根本上减少每张 GPU 上的模型参数显存。

6.3.2 all-reduce 与 ring all-reduce 直觉

all-reduce 的目标是让所有进程都拿到规约后的结果。以梯度平均为例, 假设有 K 个进程, 每个进程有本地梯度 g_i , DDP 需要得到

$$\bar{g} = \frac{1}{K} \sum_{i=1}^K g_i.$$

ring all-reduce 的直觉是: 把每个进程看成环上的一个节点, 梯度被切成若干块, 节点只和相邻节点通信。第一阶段 reduce-scatter, 把不同块的求和结果分散到各节点; 第二阶段 all-gather, 把这些块再广播回所有节点。这样可以避免单个中心节点成为瓶颈。

6.3.3 DistributedSampler 与 SyncBatchNorm

DistributedSampler 的作用是把数据集按 rank 切开, 使每个进程看到不同样本。每个 epoch 前通常要调用 `sampler.set_epoch(epoch)`, 否则 shuffle 顺序可能在不同 epoch 不变。BatchNorm 在 DDP 中也要注意。普通 BatchNorm 只使用本进程 mini-batch 的统计量; 如果每张 GPU 的 batch 很小, 统计量会不稳定。此时可考虑 SyncBatchNorm, 在多个进程间同步均值和方差。

6.3.4 最小 DDP 代码骨架

```

1 import os
2 import torch
3 import torch.distributed as dist
4 from torch.nn.parallel import DistributedDataParallel as DDP
5 from torch.utils.data import DataLoader, TensorDataset
6 from torch.utils.data.distributed import DistributedSampler
7 def main():
8     dist.init_process_group(backend="nccl")
9     local_rank = int(os.environ["LOCAL_RANK"])
10    torch.cuda.set_device(local_rank)
11    device = torch.device("cuda", local_rank)
12    x = torch.randn(1024, 32)
13    y = torch.randn(1024, 1)
14    dataset = TensorDataset(x, y)
15    sampler = DistributedSampler(dataset, shuffle=True)
16    loader = DataLoader(dataset, batch_size=32, sampler=sampler)
17    model = torch.nn.Sequential(
18        torch.nn.Linear(32, 64),
19        torch.nn.ReLU(),
20        torch.nn.Linear(64, 1),
21    ).to(device)
22    model = DDP(model, device_ids=[local_rank])
23    optimizer = torch.optim.AdamW(model.parameters(), lr=1e-3)
24    loss_fn = torch.nn.MSELoss()
25    for epoch in range(3):

```

```
26     sampler.set_epoch(epoch)
27     for xb, yb in loader:
28         xb = xb.to(device, non_blocking=True)
29         yb = yb.to(device, non_blocking=True)
30         optimizer.zero_grad(set_to_none=True)
31         pred = model(xb)
32         loss = loss_fn(pred, yb)
33         loss.backward()
34         optimizer.step()
35     dist.destroy_process_group()
36 if __name__ == "__main__":
37     main()
```

常见单机 4 卡启动方式如下：

```
1 torchrun --nproc_per_node=4 train_ddp.py
```

多机训练还需要指定节点数、节点 rank、master 地址和端口。面试中通常不要求背参数，但要知道 DDP 依赖进程组和 rank 通信。

6.4 FSDP: Fully Sharded Data Parallel

DDP 每张 GPU 都有完整参数、梯度和优化器状态。FSDP 的核心是把这些训练状态分片到不同 GPU 上，从而显著降低单卡显存占用。

6.4.1 FSDP 分片了什么

FSDP 通常关注三类状态：

- **参数**：模型权重不再长期完整保存在每张卡上。
- **梯度**：反向后的梯度按 shard 存放。
- **优化器状态**：Adam 的动量和方差等状态也可分片。

训练某一层时，FSDP 会在需要时 all-gather 出完整参数，计算结束后释放非本地 shard；反向传播时再 reduce-scatter 梯度。它用更多通信换更低显存。

6.4.2 FSDP 与 ZeRO

DeepSpeed ZeRO 和 FSDP 思路高度相关，都是把训练状态切分，避免每张 GPU 保存完整副本。面试中可以把 FSDP 理解为 PyTorch 原生态中的 ZeRO-3 风格方案，但二者在实现、API、生态和工程细节上不同。

方案	分片参数	分片梯度	分片优化器状态
ZeRO-1	否	否	是
ZeRO-2	否	是	是
ZeRO-3	是	是	是
FSDP	是	是	是

6.4.3 auto wrap 与混合精度

FSDP 不一定要把整个模型作为一个巨大模块包起来。auto wrap policy 可以按 Transformer block 等粒度自动包装子模块，控制 all-gather 的范围和时机。粒度太粗会导致峰值显存高，粒度太细会增加通信和调度开销。FSDP 常与混合精度配合使用：参数通信和计算可使用 fp16 或 bf16，优化器状态可保持 fp32，以兼顾速度、显存和稳定性。LLM 训练中，bf16 因指数范围更大，经常比 fp16 更省心。

面试速记卡

DDP 复制完整模型，FSDP 分片训练状态；DDP 主要提吞吐，FSDP 主要省显存。

6.5 三种并行方式

大模型训练常见并行方式包括数据并行、张量并行和流水线并行。实际 LLM 训练往往是混合并行，而不是只用一种。

6.5.1 数据并行

数据并行把 batch 切给不同 GPU。每张 GPU 有一份模型副本，处理不同数据，反向时同步梯度。DDP 就是典型数据并行。优点是简单、吞吐扩展好；缺点是每张 GPU 仍需容纳完整模型和优化器状态，模型太大时无能为力。

6.5.2 张量并行

张量并行把单个算子内部的大矩阵切开。例如 Megatron-LM 中常把 Transformer 的线性层权重按列或按行切分到多张 GPU。这样单层参数和计算被拆开，适合单层矩阵非常大的模型。张量并行的代价是算子之间需要频繁通信，对高速互联依赖强，通常更适合同一节点内的多卡。

6.5.3 流水线并行

流水线并行按层切分模型。前几层放在一组 GPU 上，后几层放在另一组 GPU 上。为了减少 GPU 空泡，会把一个大 batch 切成多个 micro-batch，让不同阶段像流水线一样同时工作。流水线并行适合层数很多、模型整体放不下单卡的场景，但要处理 pipeline bubble、调度和负载均衡问题。

并行方式	切分对象	主要解决	常见代价
数据并行	数据 batch	提升吞吐	每卡仍需完整模型
张量并行	单层矩阵或张量	单层太大	高频通信
流水线并行	模型层	模型层数太多	pipeline bubble
混合并行	数据、层、张量	超大模型训练	工程复杂度高

6.5.4 如何选择

一个常见判断是：如果模型能放进单卡，优先 DDP；如果训练状态占显存太大，考虑 FSDP 或 ZeRO；如果单层矩阵也很大，加入张量并行；如果层数太多，加入流水线并行。

6.6 混合精度 AMP

混合精度训练用低精度做大部分矩阵计算，用高精度保存关键状态。PyTorch 2.x 中常用 `torch.autocast` 和 `GradScaler`。

6.6.1 为什么能省显存和提速

fp16 或 bf16 的张量通常只占 fp32 的一半显存。现代 GPU 对低精度矩阵乘法有 Tensor Core 加速，因此混合精度通常既省显存又提速。但低精度也有风险：fp16 指数范围较小，可能发生 underflow 或 overflow；因此 fp16 训练常配合 loss scaling。bf16 的尾数精度低于 fp32，但指数范围接近 fp32，数值稳定性通常优于 fp16。

类型	优点	注意点
fp16	速度快，显存低	指数范围小，常需 GradScaler
bf16	指数范围大，更稳定	需要硬件支持，精度尾数较短
fp32	最稳定	显存和计算开销更高

6.6.2 最小 AMP 代码

```

1 import torch
2 model = torch.nn.Linear(32, 1).cuda()
3 optimizer = torch.optim.AdamW(model.parameters(), lr=1e-3)
4 scaler = torch.amp.GradScaler("cuda")
5 for xb, yb in loader:
6     xb = xb.cuda(non_blocking=True)
7     yb = yb.cuda(non_blocking=True)
8     optimizer.zero_grad(set_to_none=True)
9     with torch.autocast(device_type="cuda", dtype=torch.float16):
10         pred = model(xb)
11         loss = torch.nn.functional.mse_loss(pred, yb)

```

```
12     scaler.scale(loss).backward()
13     scaler.step(optimizer)
14     scaler.update()
```

如果使用 bf16, 很多场景可以不使用 GradScaler:

```
1  with torch.autocast(device_type="cuda", dtype=torch.bfloat16):
2      pred = model(xb)
3      loss = loss_fn(pred, yb)
```

6.7 显存优化常见手段

6.7.1 梯度累积

梯度累积用多个小 mini-batch 累积梯度, 再执行一次 optimizer step, 从而模拟更大的 global batch。若累积步数为 A , 每卡 batch 为 B , 数据并行进程数为 K , 则有效 batch size 约为

$$B_{\text{global}} = A \times B \times K.$$

```
1  accum_steps = 4
2  optimizer.zero_grad(set_to_none=True)
3  for step, (xb, yb) in enumerate(loader):
4      with torch.autocast(device_type="cuda", dtype=torch.bfloat16):
5          pred = model(xb.cuda())
6          loss = loss_fn(pred, yb.cuda()) / accum_steps
7      loss.backward()
8      if (step + 1) % accum_steps == 0:
9          optimizer.step()
10         optimizer.zero_grad(set_to_none=True)
```

关键点是 loss 要除以累积步数, 否则梯度尺度会变大。DDP 中还可结合 `no_sync()` 减少非 step 轮次的梯度同步开销。

6.7.2 梯度检查点

梯度检查点也叫 activation checkpointing。普通反向传播会保存中间激活值; 检查点技术只保存部分节点, 反向时重新计算被丢弃的激活值。它用额外计算换显存。适用场景是 Transformer 层数深、激活值占用很高的模型。代价是训练速度下降, 因为反向阶段需要重算部分前向。

6.7.3 参数 offload

offload 是把部分参数、梯度或优化器状态放到 CPU 内存甚至 NVMe 中, 需要时再搬回 GPU。它能进一步降低 GPU 显存压力, 但会引入 PCIe 或存储 IO 开销。面试中可回答: offload 适合显存极紧张时保命, 不是无成本加速。

6.8 高频面试题

面试题

DDP 为什么通常比 DP 好?

参考答案

DP 是单进程多线程, 主 GPU 负责 gather 和更新, 容易受 GIL、复制开销和主卡瓶颈影响。DDP 是多进程, 通常每 GPU 一个进程, 梯度通过 all-reduce 同步, 负载更均衡, 通信计算可重叠, 也支持多机扩展, 因此是生产训练首选。

面试题

all-reduce 是什么? DDP 为什么需要它?

参考答案

all-reduce 是一种集合通信操作, 把多个进程上的张量先做规约, 例如求和或平均, 再把结果发回所有进程。DDP 中每个进程处理不同数据, 得到本地梯度; 为了让模型副本保持一致, 需要对梯度做 all-reduce, 然后每个进程用相同的平均梯度更新参数。

面试题

FSDP 和 ZeRO 是什么关系?

参考答案

二者都通过分片训练状态降低单卡显存。ZeRO-1 分片优化器状态, ZeRO-2 进一步分片梯度, ZeRO-3 分片参数、梯度和优化器状态。FSDP 也是参数、梯度、优化器状态分片的思路, 可理解为 PyTorch 原生生态中的 ZeRO-3 风格方案, 但实现和使用接口不同。

面试题

数据并行、张量并行、流水线并行的区别是什么?

参考答案

数据并行切 batch, 每张卡有完整模型, 主要提升吞吐。张量并行切单层矩阵或张量, 解决单层太大的问题, 但通信频繁。流水线并行按层切模型, 通过 micro-batch 让不同阶段同时工作, 解决模型层数多、整体放不下的问题, 但会有 pipeline bubble。

面试题

如果一个大模型放不下单卡, 训练时有哪些方案?

参考答案

先区分瓶颈。如果是优化器状态和梯度太大, 可用 FSDP 或 ZeRO。若单层矩阵过大, 可加张量并行。若层数太多, 可用流水线并行。还可以配合 AMP、梯度检查点、梯度累积和 offload。实际 LLM 训练常用数据并行、张量并行、流水线并行和分片技术的混合方案。

面试题

梯度累积如何模拟大 batch?

参考答案

把一个大 batch 拆成多个小 mini-batch, 连续做前向和反向但暂不更新参数, 累积若干次梯度后再执行一次 optimizer step。为了保持梯度尺度一致, 通常把每次 loss 除以累积步数。有效 batch size 等于每卡 batch、累积步数和数据并行进程数的乘积。

面试题

fp16 和 bf16 有什么区别?

参考答案

二者都比 fp32 省显存, 适合混合精度训练。fp16 尾数相对更多但指数范围小, 容易 overflow 或 underflow, 通常需要 GradScaler。bf16 指数范围接近 fp32, 数值稳定性更好, LLM 训练中更常见, 但需要硬件支持, 且尾数精度较低。

6.9 章末速记卡

面试速记卡

DDP: 每 GPU 一个进程, 每进程一份完整模型, 反向时 all-reduce 梯度; 比 DP 更快、更均衡、更适合多机。

面试速记卡

FSDP: 分片参数、梯度和优化器状态, 用通信换显存; 适合训练单卡放不下训练状态的大模型。

面试速记卡

三种并行: 数据并行切 batch, 张量并行切矩阵, 流水线并行切层; 超大模型常用混合并行。

面试速记卡

AMP: 用 fp16/bf16 做主要计算以省显存和提速; fp16 常配 GradScaler, bf16 通常更稳定。

Chapter 7

速查表与常见坑

7.1 常用张量操作速查

本章像一张考前小抄：先把高频 API、输入输出形状和训练流程压成表格，再集中列出最容易在面试和项目中踩到的坑。阅读时建议边看边问自己三个问题：张量形状是什么、梯度图还在不在、数据和模型是否在同一设备上。

7.1.1 创建、形状、索引、拼接与规约

类别	常用写法	要点
从数据创建	<code>torch.tensor(data)</code>	会复制数据；可指定 <code>dtype</code> 和 <code>device</code> 。
未初始化	<code>torch.empty(2, 3)</code>	只分配内存，不保证初值，调试时不要当作零张量。
全零/全一	<code>torch.zeros(2, 3)</code> , <code>torch.ones_like(x)</code>	<code>_like</code> 版本继承形状，通常也继承 <code>dtype</code> 和 <code>device</code> 。
随机数	<code>torch.randn(4, 5)</code> , <code>torch.randint(0, 10, (3,))</code>	<code>randn</code> 是标准正态； <code>rand</code> 是 $[0, 1)$ 均匀分布。
区间序列	<code>torch.arange(0, 10, 2)</code> , <code>torch.linspace(0, 1, 5)</code>	<code>arange</code> 类似 Python <code>range</code> ，右端通常不包含。
形状查看	<code>x.shape</code> , <code>x.size()</code> , <code>x.dim()</code>	<code>shape</code> 是属性， <code>size()</code> 返回 <code>torch.Size</code> 。
类型转换	<code>x.float()</code> , <code>x.long()</code> , <code>x.to(dtype=torch.float32)</code>	分类标签常用 <code>long</code> ；回归输入常用浮点。
索引切片	<code>x[:, 0]</code> , <code>x[mask]</code> , <code>x[idx]</code>	布尔索引和高级索引通常会产生新张量。
拼接	<code>torch.cat([a, b], dim=0)</code>	只在指定维度上大小可不同，其他维度必须相同。
新增维拼接	<code>torch.stack([a, b], dim=0)</code>	会新增一个维度，所有输入形状必须完全相同。
规约	<code>x.sum(dim=1)</code> , <code>x.mean()</code> , <code>x.max(dim=1)</code>	<code>max(dim)</code> 返回值和索引两个张量。
保留维度	<code>x.sum(dim=1, keepdim=True)</code>	便于后续广播，减少手动 <code>unsqueeze</code> 。

7.1.2 形状操作对照

操作	示例	记忆点
view	<code>x.view(batch, -1)</code>	要求内存连续；-1 表示自动推断。
reshape	<code>x.reshape(batch, -1)</code>	更宽松；必要时可能复制数据。
permute	<code>x.permute(0, 2, 3, 1)</code>	任意重排维度，常用于 NCHW 与 NHWC 转换。
transpose	<code>x.transpose(1, 2)</code>	只交换两个维度，是 <code>permute</code> 的常见特例。
squeeze	<code>x.squeeze(1)</code>	删除大小为 1 的维度；建议指定 <code>dim</code> 。
unsqueeze	<code>x.unsqueeze(0)</code>	插入大小为 1 的维度，常用于补 batch 维。
flatten	<code>torch.flatten(x, 1)</code>	从指定维开始展平，CNN 接全连接层常用。
expand	<code>x.expand(4, 3)</code>	共享存储的广播视图，不真正复制。
repeat	<code>x.repeat(4, 1)</code>	真正复制数据，显存开销更大。

```
1 # CNN 特征接 Linear 的典型写法
2 x = torch.flatten(x, start_dim=1) # [N, C, H, W] -> [N, C*H*W]
3 logits = classifier(x)
```

7.2 模型层、损失、优化器与调度器

7.2.1 常用层速查

层	常见输入/输出	面试要点
<code>nn.Linear</code>	输入 [..., in_features], 输出 [..., out_features]	只作用在最后一维; 参数量为 <code>in_features * out_features + out_features</code> 。
<code>nn.Conv2d</code>	[N,C,H,W] 到 [N,C_out,H_out, W_out]	关注 <code>kernel_size</code> 、 <code>stride</code> 、 <code>padding</code> 、 <code>groups</code> 。
<code>nn.BatchNorm2d</code>	常接在卷积后，输入 [N,C,H,W]	训练时用 batch 统计量，推理时用 running mean/var。
<code>nn.LayerNorm</code>	常用于 Transformer，按最后若干维归一化	不依赖 batch 统计量，小 batch 更稳定。
<code>nn.Dropout</code>	训练时随机置零，推理时恒等映射	受 <code>model.train()</code> 和 <code>model.eval()</code> 控制。
<code>nn.Embedding</code>	输入整型索引，输出 dense 向量	输入 dtype 应为 <code>torch.long</code> ；本质是查表。

7.2.2 损失函数对照

7.2.3 优化器与调度器对照

```
1 for x, y in loader:
2     x, y = x.to(device), y.to(device)
3     optimizer.zero_grad()
4     loss = criterion(model(x), y)
```

损失	用途	输入要求
<code>nn.CrossEntropyLoss</code>	单标签多分类	输入 raw logits, 形状 $[N, C]$; 标签为类别下标 $[N]$, dtype 为 <code>long</code> 。
<code>nn.BCEWithLogitsLoss</code>	二分类或多标签分类	输入 raw logits; 标签通常为同形状浮点 0/1。内部已包含 sigmoid。
<code>nn.MSELoss</code>	回归、重建	预测和目标形状相同, 均为浮点。
<code>nn.L1Loss</code>	对异常值更鲁棒的回归	优化绝对误差, 梯度不像 MSE 那样随误差线性增大。
<code>nn.KLDivLoss</code>	分布匹配、蒸馏	默认输入是 log probability; 常配合 <code>log_softmax</code> 。
<code>nn.CosineEmbeddingLoss</code>	表征相似度学习	输入两个向量和标签 $1/-1$ 。

组件	典型场景	记忆点
<code>optim.SGD</code>	经典 CNN、需要强正则的任务	常配合 momentum; 泛化常被认为较稳。
<code>optim.Adam</code>	默认首选、收敛快	自适应学习率; 注意合理设置 <code>weight_decay</code> 。
<code>optim.AdamW</code>	Transformer、预训练微调	将权重衰减与梯度更新解耦, 是现代模型常用默认项。
<code>StepLR</code>	固定轮数衰减	每隔 <code>step_size</code> 个 epoch 乘以 <code>gamma</code> 。
<code>CosineAnnealingLR</code>	训练预算固定时	学习率按余弦曲线下降, 常有更平滑的后期训练。
<code>ReduceLROnPlateau</code>	监控验证集指标	调用 <code>scheduler.step(val_loss)</code> , 不是简单每轮 step。
<code>OneCycleLR</code>	需要快速训练时	通常按 iteration step, 而不是按 epoch step。

```
5     loss.backward()
6     optimizer.step()
```

7.3 设备迁移、保存加载与 API 对照

7.3.1 设备迁移与保存加载

任务	推荐写法	注意事项
选择设备	<code>device = torch.device("cuda" if torch.cuda.is_available() else "cpu")</code>	不要在代码中硬编码必须有 GPU。
迁移模型	<code>model.to(device)</code>	迁移后新建的张量也要放到同一设备。
迁移数据	<code>x = x.to(device)</code>	<code>to</code> 通常返回新张量，别忘记接收返回值。
保存参数	<code>torch.save(model.state_dict(), path)</code>	最推荐，文件更稳定，不依赖完整类定义的 pickle。
加载参数	<code>model.load_state_dict(torch.load(path, map_location=device))</code>	先实例化同结构模型，再加载参数。
保存检查点	<code>torch.save({"model": model.state_dict(), "optim": optimizer.state_dict()}, path)</code>	继续训练时同时保存 epoch、优化器、调度器和随机状态更稳。
切换推理	<code>model.eval()</code> 与 <code>torch.no_grad()</code>	前者影响层行为，后者关闭梯度记录，二者不是一回事。

7.3.2 NumPy 与 Torch API 对照

7.4 常见坑与排查方式

常见坑

忘记 `optimizer.zero_grad()`。PyTorch 默认会累加梯度，不清零会把多个 batch 的梯度叠在一起，等价于错误的梯度累积。标准顺序是 `zero_grad()`、前向、`backward()`、`step()`；如果刻意做梯度累积，要按累积步数清零并缩放 loss。

常见坑

推理时忘记 `model.eval()` 或 `torch.no_grad()`。`eval()` 会让 BatchNorm 使用运行统计量、Dropout 停止随机置零；`no_grad()` 会关闭计算图记录，节省显存并加速。验证和测试通常两者都要写。

NumPy	PyTorch	差异提示
<code>np.array(data)</code>	<code>torch.tensor(data)</code>	PyTorch 张量可带梯度和设备信息。
<code>np.asarray(a)</code>	<code>torch.as_tensor(a)</code>	可能共享底层内存，修改时要小心。
<code>np.zeros(shape)</code>	<code>torch.zeros(shape)</code>	PyTorch 可直接指定 <code>device</code> 。
<code>np.random.randn(m,n)</code>	<code>torch.randn(m, n)</code>	随机种子分别由 NumPy 和 Torch 管理。
<code>a.reshape(...)</code>	<code>x.reshape(...)</code>	PyTorch 还要考虑梯度和连续性。
<code>np.transpose(a, axes)</code>	<code>x.permute(*dims)</code>	两者都只是重排维度语义。
<code>np.concatenate(list, axis)</code>	<code>torch.cat(list, dim)</code>	参数名从 <code>axis</code> 变为 <code>dim</code> 。
<code>np.stack(list, axis)</code>	<code>torch.stack(list, dim)</code>	都会新增维度。
<code>np.sum(a, axis)</code>	<code>x.sum(dim)</code>	PyTorch 常用 <code>keepdim</code> 保持维度。
<code>a.astype(np.float32)</code>	<code>x.to(torch.float32)</code> 或 <code>x.float()</code>	PyTorch 的 <code>to</code> 也可迁移设备。
<code>a @ b</code>	<code>x @ y</code> 或 <code>torch.matmul(x, y)</code>	批量矩阵乘法会按高维广播。

```

1 model.eval()
2 with torch.no_grad():
3     logits = model(x.to(device))

```

常见坑

累加 loss 时未 `.item()` 或未 `detach()`。如果直接 `total_loss += loss`，可能把每个 batch 的计算图都挂住，导致显存持续增长。用于日志时写 `total_loss += loss.item()`；需要保留张量但不要梯度时用 `loss.detach()`。

常见坑

CrossEntropyLoss 前又手动 softmax。`nn.CrossEntropyLoss` 内部已经包含 `log_softmax` 和负对数似然。输入应是 raw logits；手动 softmax 会让数值稳定性变差，也可能让梯度表现异常。

常见坑

view 前忘记 `contiguous()`。`permute`、`transpose` 后得到的张量通常不是连续内存，直接 `view` 可能报错。可写 `x = x.contiguous().view(...)`，或者更常用 `x = x.reshape(...)`。

常见坑

device 不一致：CPU 与 CUDA 混用。常见报错是模型在 GPU、输入或标签还在 CPU，或者新建的 mask、位置编码默认在 CPU。排查时打印 `x.device`、`y.device` 和 `next(model.parameters()).device`。

常见坑

Windows 下 DataLoader num_workers 的注意事项。 Windows 使用 spawn 启动子进程，训练脚本应放在 `if __name__ == "__main__":` 保护下。调试阶段可先设 `num_workers=0`；确认数据集可 pickle 后再增大 worker 数。

常见坑

随机种子与可复现只设一个还不够。 通常要同时设置 Python、NumPy、Torch 和 CUDA 随机种子，并处理 cuDNN 的确定性选项。注意确定性设置可能降低速度，而且某些算子仍可能不可完全复现。

```

1 import random
2 import numpy as np
3 import torch
4
5 seed = 42
6 random.seed(seed)
7 np.random.seed(seed)
8 torch.manual_seed(seed)
9 torch.cuda.manual_seed_all(seed)
10 torch.backends.cudnn.deterministic = True
11 torch.backends.cudnn.benchmark = False

```

常见坑

in-place 操作破坏 autograd。 带下划线的操作如 `relu_()`、`add_()` 会原地修改张量。如果某个中间值还被反向传播需要，原地修改会触发版本号错误；不确定时优先使用非原地写法。

常见坑

广播导致维度静默出错。 例如预测 `[N,1]` 与标签 `[N]` 做 MSE，可能广播成 `[N,N]` 而不立刻报错。训练前断言关键形状：`assert pred.shape == target.shape`，尤其是回归、多标签和 mask 场景。

常见坑

保存整个模型而不是只存 state_dict。 `torch.save(model, path)` 依赖 Python pickle 和类路径，代码结构一改就可能加载失败。生产和面试推荐回答：保存 `state_dict`，加载时重新构造模型，再 `load_state_dict`。

7.5 临场速记：面试前五分钟

1. 训练四步：`zero_grad`、`forward`、`backward`、`step`。
2. 验证两步：`model.eval()` 加 `with torch.no_grad():`。

3. 分类 logits 直接喂 `CrossEntropyLoss`，不要先 `softmax`。
4. 二分类或多标签优先用 `BCEWithLogitsLoss`，不要手动 `sigmoid` 后再喂它。
5. `cat` 是沿已有维拼接，`stack` 是新增维拼接。
6. `view` 要连续，`reshape` 更省心；`permute` 后尤其要注意。
7. `squeeze` 最好指定维度，避免 batch size 为 1 时把 batch 维挤掉。
8. 新建张量、mask、标签和模型参数必须在同一 device。
9. 日志 loss 用 `loss.item()`，不要把计算图存进列表。
10. BatchNorm 和 Dropout 的行为由 `train()` / `eval()` 控制。
11. 保存模型优先保存 `state_dict`，恢复训练还要保存优化器和 epoch。
12. 可复现要同时管 Python、NumPy、Torch、CUDA 和 cuDNN。

7.6 章末 quickcard

面试速记卡

PyTorch 速查口诀

- 形状先行：每经过一层都能说出 batch、channel、sequence 和 feature 维。
- 梯度清楚：训练开图，验证关图；累加数值用 `item()`。
- 损失匹配：多分类 logits 配 CE，多标签 logits 配 `BCEWithLogits`。
- 设备统一：模型、输入、标签、临时张量都在同一个 device。
- 保存稳妥：少存整个对象，多存 `state_dict` 和必要训练状态。
- 排坑优先级：shape、dtype、device、train/eval、requires_grad。

Chapter 8

面试打法与冲刺路线

8.1 本章定位：把知识变成可录用的表达

前面几章解决的是「会不会」：张量、自动求导、训练循环、模型结构、优化调参与工程部署。本章解决的是「能不能在面试中讲清楚」。同样一个项目，有人只说「我用了 ResNet，准确率不错」；有人能讲清业务目标、数据来源、模型选择、指标取舍、上线风险和复盘改进。后者更容易让面试官相信：你不仅会调包，也能把模型放进真实问题里。面试表达的核心不是背答案，而是建立稳定框架。本章围绕三类场景展开：

- 项目面试：把项目讲成「有问题、有方案、有指标、有复盘」的故事。
- 系统设计：用简版机器学习系统设计框架回答开放题。
- 行为面试与冲刺路线：把本书知识转化为可输出的话术。

8.2 项目怎么讲：从流水账到结构化叙事

8.2.1 STAR 框架

STAR 是项目面试和行为面试都适用的叙事结构：

- **S (Situation, 背景)**：项目发生在什么场景中，为什么要做。
- **T (Task, 任务)**：你负责什么，目标和约束是什么。
- **A (Action, 行动)**：你如何分析数据、选择模型、训练调优、排查问题。
- **R (Result, 结果)**：指标提升、业务收益、工程效果和复盘结论。

面试速记卡

项目开场句式：「这个项目的目标是解决 _____ 问题。我主要负责 _____。我们使用 _____ 数据，核心模型是 _____，最终在 _____ 指标上从 _____ 提升到 _____。」

8.2.2 问题—数据—模型—指标—优化—上线

深度学习项目最好不要只按时间顺序讲，更稳妥的方式是按工程闭环讲：

1. **问题**：业务目标是什么，输入输出是什么，错误代价是什么。
2. **数据**：数据来自哪里，规模如何，标签如何获得，噪声在哪里。
3. **模型**：为什么选这个模型，baseline 是什么，复杂度是否匹配。
4. **指标**：离线指标和线上指标分别是什么，是否存在冲突。
5. **优化**：做了哪些调参、数据增强、损失函数、采样或工程优化。
6. **上线**：如何部署、监控、回滚、迭代，如何处理数据漂移。

这个框架适合回答「介绍一个你最熟悉的项目」。面试官可以沿着每一层继续追问，而你的回答不会散。

8.2.3 把一个深度学习项目讲清楚的模板

面试题

请介绍一个你做过的深度学习项目。

参考答案

可以按下面模板回答：

1. **背景与目标**：这个项目要解决什么问题，为什么传统规则或浅层模型不够。
2. **数据与标签**：数据规模、类别分布、标注方式、训练验证测试划分。
3. **模型与 baseline**：先做了什么简单 baseline，再选择了什么深度模型。
4. **训练细节**：损失函数、优化器、学习率策略、batch size、正则化与增强。
5. **评估指标**：主指标、辅助指标、错误样例分析。
6. **问题与优化**：过拟合、类别不平衡、训练不稳定、推理延迟等问题如何解决。
7. **结果与复盘**：指标提升、业务影响、上线监控、下一步计划。

8.2.4 示例话术：图像缺陷分类项目

参考答案

我做过一个工业图像缺陷分类项目，目标是根据产线相机拍摄的图片判断产品是否存在划痕、污点和边缘破损。业务上希望减少人工复检压力，同时不能漏掉严重缺陷，所以召回率比单纯准确率更重要。数据方面，我们一开始有约 8 万张图片，其中正常样本占比接近 80%，缺陷类别明显不均衡。我先清洗重复图和曝光异常图，再按产线批次划分训练集、验证集和测试

集，避免同一批产品同时出现在训练和测试中导致指标虚高。模型方面，我先用传统特征加 SVM 做 baseline，之后尝试 ResNet 系列，并用 ImageNet 预训练权重初始化。选择 ResNet 的原因是数据量中等、缺陷形态局部明显，残差结构训练稳定，工程上也容易部署。训练时使用交叉熵损失，针对类别不平衡加入 class weight，并使用随机裁剪、颜色扰动和轻微旋转做增强。优化器使用 AdamW，先冻结 backbone 训练分类头，再整体微调。主指标选择缺陷类 recall 和 macro F1，而不是只看 accuracy。主要问题是模型对反光区域误判较多。我通过错误样例分析发现这类样本覆盖不足，于是补充反光正常样本，并增加亮度扰动。最终缺陷召回率从 baseline 的 86% 提升到 94%，macro F1 提升到 0.91。上线前我们做了阈值调节和人工复核兜底，线上持续监控各类别分布和误报率。

8.2.5 常见追问与回答要点

面试题

为什么选择这个模型，而不是更复杂的模型？

参考答案

不要只说「效果好」。可以从数据规模、任务特征、训练稳定性和部署成本回答：「我先做简单 baseline，再比较 ResNet 和更大的 ViT。在我们的数据规模下，ResNet 已经能达到接近的验证集 F1，同时推理延迟更低、部署链路更成熟，所以最终选择它作为上线模型。」

面试题

为什么选择这个指标？

参考答案

指标要和业务代价绑定。漏检代价高，就强调 recall；误报影响用户体验，就强调 precision；类别不均衡，就说明为什么不用 accuracy 作为唯一指标。最好同时说明离线指标和线上指标的关系。

面试题

你是如何调优的？项目中遇到过什么困难？

参考答案

调优建议按「先数据、再模型、再训练、最后阈值」回答。常见动作包括清洗标签、处理类别不平衡、数据增强、学习率搜索、冻结与微调、正则化、早停、阈值移动、错误样例回流。困难要选择真实且有技术含量的问题，例如标签噪声、数据泄漏、线上分布漂移、训练不稳定、显存不足或推理延迟。回答顺序是：现象、定位、措施、结果、复盘。

常见坑

项目面试的大坑是只讲模型名字。面试官真正想知道的是：你是否理解数据、指标和约束；是否能解释每个技术选择；是否能从错误中迭代，而不是只复述论文或 API。

8.3 简版机器学习系统设计答法

8.3.1 面试中的七个模块

机器学习系统设计题通常不会要求完整工业级架构。面试官更关心你是否有端到端意识。推荐按下面七个模块组织：

1. **需求**：输入、输出、用户、延迟、吞吐、准确性、可解释性和安全要求。
2. **数据与特征**：数据来源、采集方式、标签、特征处理、训练服务一致性。
3. **模型选型**：baseline、候选模型、复杂度、冷启动方案。
4. **训练管线**：样本构建、训练验证划分、调参、版本管理、实验追踪。
5. **评估指标**：离线指标、线上指标、A/B 测试、分桶评估。
6. **部署与监控**：批处理或在线服务、延迟、扩缩容、模型回滚、日志。
7. **迭代**：数据回流、漂移检测、主动学习、灰度发布。

8.3.2 例子一：设计一个图像分类服务

面试题

如果让你设计一个商品图片分类服务，你会怎么做？

参考答案

1. **需求**：输入商品图片，输出一级或多级类目；支持在线上传识别，延迟可控。
2. **数据**：使用历史图片和人工审核类目作为标签；清洗重复图、低质量图和错误标签；按商家或时间切分测试集。
3. **模型**：先用预训练 CNN 或 ViT 做 baseline；如果类目层级明显，可做层级分类或多头输出。
4. **训练**：使用交叉熵损失，处理类别不平衡；加入裁剪、颜色扰动等增强；记录模型版本和数据版本。
5. **评估**：离线看 top-1 accuracy、top-5 accuracy、macro F1；重点类目单独看召回率。
6. **部署**：导出为推理服务；图片先 resize 和归一化；设置超时、降级和人工审核兜底。
7. **监控**：监控请求量、延迟、错误率、预测分布、低置信度样本比例；定期用新标注数据再训练。

8.3.3 例子二：设计一个推荐打分服务

面试题

如果让你设计一个推荐系统中的候选物品打分服务，你会怎么答？

参考答案

1. **需求**：输入用户、上下文和候选物品，输出点击率或转化率预估分数；在线延迟要低。
2. **数据与特征**：用户历史行为、物品属性、上下文特征；注意曝光未点击样本、位置偏差和时间穿越。
3. **模型**：先用 LR 或 GBDT 做 baseline，再考虑 Wide & Deep、DIN、双塔或序列兴趣模型。
4. **训练管线**：按时间切分训练和验证；特征离线生成并保证在线一致；负采样策略要和业务曝光逻辑一致。
5. **评估**：离线看 AUC、logloss、校准误差；线上看 CTR、CVR、停留时长和用户留存。
6. **部署**：高频特征放缓存，模型服务做批量打分；对超时请求使用轻量模型或热门结果降级。
7. **迭代**：监控特征缺失率、分数分布、线上离线差异；通过 A/B 测试逐步放量。

常见坑

系统设计题不要一上来就报模型名。先澄清需求，再谈数据，再谈模型。没有数据闭环，再先进的模型也只是孤立组件。

8.4 行为面试：用 STAR 回答软技能问题

行为面试考察合作方式、抗压能力、复盘能力和职业成熟度。技术同学容易忽视这一部分，但它常常影响最终录用。

8.4.1 常见问题与答法要点

8.4.2 行为题示例话术

面试题

请讲一次你遇到困难并最终解决的经历。

参考答案

「在一个图像分类项目中，模型离线准确率较高，但上线灰度后误报率明显高于测试集。我负责定位这个问题。我先按时间、设备和场景切分线上样本，发现夜间低光场景的错误占比最

常见问题	STAR 答法要点
讲一次你解决复杂问题的经历	背景具体，行动体现拆解、验证和复盘，结果最好有指标。
讲一次你与他人意见不一致	不贬低对方；强调用数据、实验或用户价值对齐目标。
讲一次失败经历	选择可控失败；说明当时判断、后续补救和机制改进。
你如何学习新技术	给出路径：文档、最小 demo、源码或论文、项目落地、复盘输出。
你如何处理 deadline 压力	说明优先级、风险沟通、范围裁剪和阶段性交付。
为什么想做深度学习岗位	连接个人经历、能力积累、岗位需求和长期目标。

高；随后回查训练集，发现这类样本覆盖不足，而且数据增强没有模拟低光条件。我补充采集低光样本，加入亮度扰动，并把测试集按场景分桶评估。调整后整体 F1 提升不大，但低光场景误报率下降约 30%。这件事让我意识到，平均指标不足以代表线上表现，关键场景必须单独监控。」

面试速记卡

行为面试不要讲成「我很努力」。要讲「我发现了什么问题、做了什么判断、采取了什么行动、产生了什么结果、沉淀了什么方法」。

8.5 冲刺学习路线

8.5.1 3 天速成计划

3 天计划适合已有 Python 和机器学习基础、临近面试需要快速激活知识的人。

天数	对应章节	目标产出
Day 1	第 1-2 章：张量、自动求导、训练循环	能手写训练循环，讲清 tensor、grad、loss、optimizer、反向传播。
Day 2	第 3-5 章：网络模块、CNN、序列与 Transformer	能解释常见层、过拟合、卷积、注意力和模型选择理由。
Day 3	第 6-8 章：优化调参、工程部署、面试打法	准备项目话术，背熟系统设计七模块和章末 quickcard。

- 1. **Day 1：打牢训练闭环。**复习 tensor 形状、广播、索引、自动求导；手写最小训练循环；准备回答「反向传播」「梯度清零」「train/eval 区别」。
- 2. **Day 2：补齐模型知识。**复习 nn.Module、loss、optimizer、CNN、RNN、attention；每个模型准备一句适用场景和一句局限性。

3. **Day 3: 转成面试表达。**用本章模板整理一个项目；练习图像分类或推荐打分服务的系统设计；整理 10 个高频问答并口述演练。

8.5.2 7 天速成计划

7 天计划适合基础一般但有一周准备时间的读者。建议每天安排一轮看书和一轮口述。

1. **Day 1: PyTorch 基础。**对应第 1 章。掌握 tensor 创建、dtype、device、view/reshape、广播、索引和常见 API。
2. **Day 2: 自动求导与训练循环。**对应第 2 章。掌握计算图、requires_grad、backward、no_grad、梯度累积；能白板写训练循环。
3. **Day 3: 网络模块与损失函数。**对应第 3 章。复习 nn.Module、Sequential、参数注册、初始化、常见 loss。
4. **Day 4: CNN 与视觉任务。**对应第 4 章。掌握卷积、池化、BatchNorm、ResNet、迁移学习和数据增强；准备图像分类项目话术。
5. **Day 5: 序列模型与 Transformer。**对应第 5 章。掌握 embedding、RNN/LSTM、attention、Transformer；能解释 self-attention。
6. **Day 6: 优化、调参与工程问题。**对应第 6–7 章。复习 Adam/SGD、学习率、过拟合、显存优化、混合精度、部署监控。
7. **Day 7: 模拟面试。**对应第 8 章。完成一次项目自述、一次 ML 系统设计、一次行为面试；准备 30 秒、2 分钟、5 分钟版本。

8.6 资源清单：少而精

8.6.1 官方文档与关键 API

- PyTorch 官方文档：重点看 Tensor、Autograd、nn、optim、DataLoader、torchvision。
- PyTorch Tutorials：复习训练循环、迁移学习、保存加载模型。
- 关键 API：reshape、view、permute、cat、stack、requires_grad、backward、detach、no_grad、zero_grad。
- 模型与训练：nn.Module、forward、parameters、state_dict、CrossEntropyLoss、MSELoss、SGD、Adam、AdamW。
- 工程能力：model.train()、model.eval()、torch.save、torch.load、cuda、autocast、GradScaler。

8.6.2 刷题方向

- 概念题：反向传播、过拟合、正则化、归一化、优化器区别。
- 代码题：手写 Dataset、训练循环、冻结层、保存加载模型、计算准确率。
- 项目题：准备一个视觉或 NLP 项目，能讲数据、模型、指标和问题定位。
- 系统题：图像分类服务、推荐排序服务、文本分类服务、异常检测服务。
- 行为题：失败经历、冲突处理、学习新技术、项目推进、压力管理。

8.7 与《AI Agent 工程师转行与面试完全指南》的衔接

同仓库中的《AI Agent 工程师转行与面试完全指南》更偏应用层，而本书更偏模型层和 PyTorch 实践层。两者不是替代关系，而是上下游关系。

- **模型层 (本书)**：关注张量、训练、损失函数、优化、神经网络结构、模型评估与部署基础。
- **应用层 (AI Agent 书)**：关注大模型应用、工具调用、RAG、记忆、规划、Agent workflow、工程集成和产品落地。
- **互补关系**：本书回答「模型怎么来」，AI Agent 书回答「模型怎么用」。

如果目标岗位是深度学习工程师、算法工程师或计算机视觉工程师，建议先读本书，重点打牢训练与模型基础。如果目标岗位是 AI 应用工程师、Agent 工程师或大模型产品工程师，建议在掌握本书第 1、2、6、7 章基础后，转向 AI Agent 书强化应用系统能力。如果岗位要求「既会训练又会落地」，两本书应结合使用。

8.8 章末 quickcard：面试当天行动清单

面试速记卡

1. 提前准备 1 个主项目和 1 个备选项目，每个都有 30 秒、2 分钟、5 分钟版本。
2. 项目介绍按「问题-数据-模型-指标-优化-上线」讲，不要只背模型名。
3. 所有指标都要能解释业务含义：为什么看它，它的缺点是什么。
4. 遇到不会的问题，先说理解，再拆解假设，不要沉默或硬编。
5. 系统设计先澄清需求，再讲数据、模型、训练、评估、部署和监控。
6. 行为题坚持 STAR：背景简短，行动具体，结果量化，复盘真诚。
7. 代码题先保证正确性，再谈效率；注意 tensor shape、device 和梯度清零。
8. 面试前复习高频 API：Dataset、DataLoader、nn.Module、optimizer、loss、state_dict。
9. 准备 2-3 个反问：团队任务、数据规模、模型上线方式、评估与迭代流程。

10. 面试后立即复盘：记录没答好的问题，补到自己的题库里。